

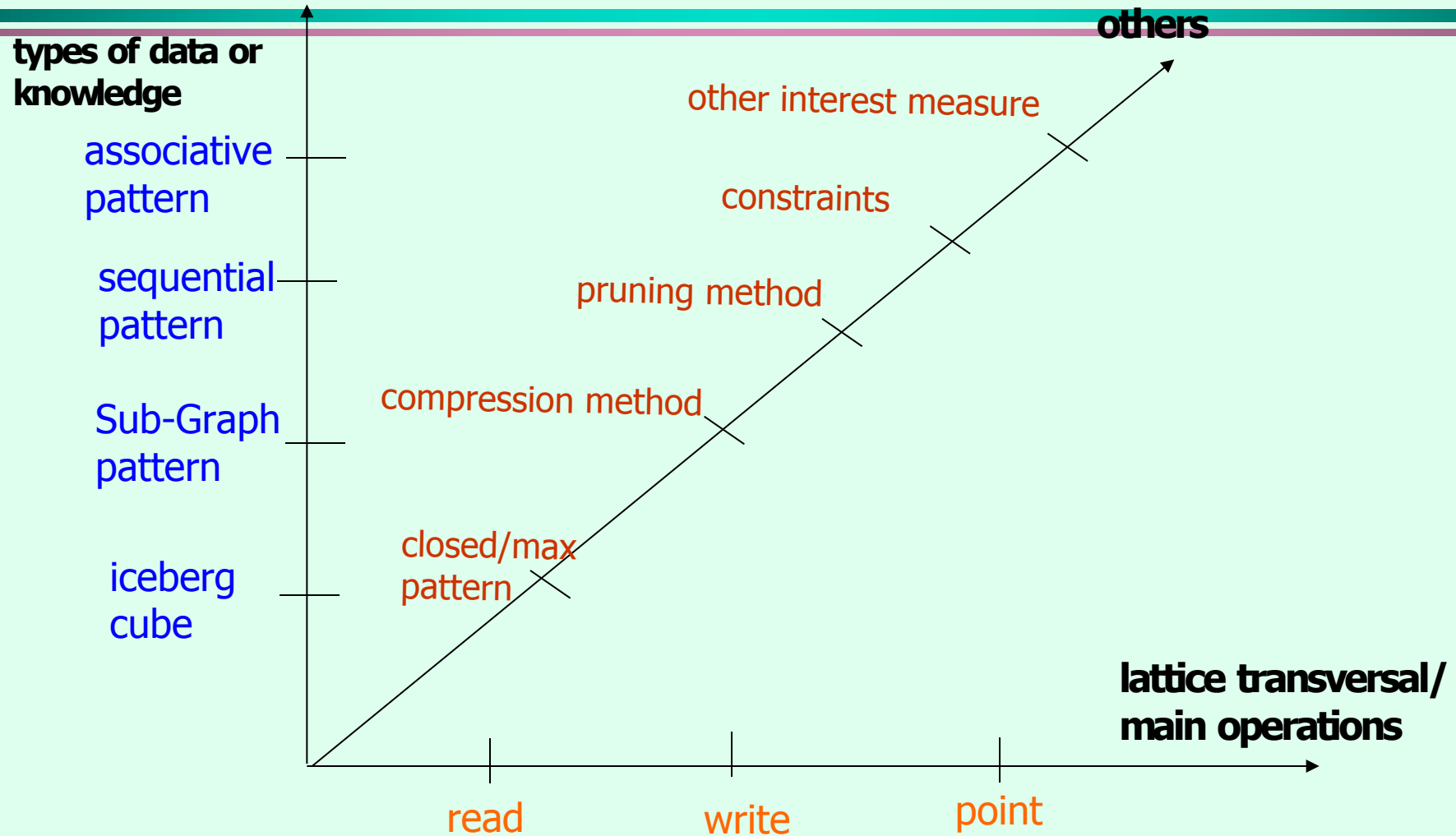
What Is Frequent Pattern Analysis?

- ✓ **Frequent pattern:** a pattern (a set of items, subsequences, substructures, etc.) that occurs frequently in a data set
- ✓ First proposed by Agrawal, Imielinski, and Swami [AIS93] in the context of **frequent itemsets** and **association rule mining**
- ✓ **Motivation: Finding inherent regularities in data**
 - u What products were often purchased together?— Beer and diapers?!
 - u What are the subsequent purchases after buying a PC?
 - u What kinds of DNA are sensitive to this new drug?
 - u Can we automatically classify web documents?
- ✓ **Applications**
 - u Basket data analysis, cross-marketing, catalog design, sale campaign analysis, Web log (click stream) analysis, and DNA sequence analysis.

Why Is Freq. Pattern Mining Important?

- ✓ **Discloses an intrinsic and important property of data sets**
- ✓ **Forms the foundation for many essential data mining tasks**
 - u **Association, correlation, and causality analysis**
 - u **Sequential, structural (e.g., sub-graph) patterns**
 - u **Pattern analysis in spatiotemporal, multimedia, time-series, and stream data**
 - u **Classification: associative classification**
 - u **Cluster analysis: frequent pattern-based clustering**
 - u **Data warehousing: iceberg cube and cube-gradient**
 - u **Semantic data compression: fascicles**
 - u **Broad applications**

A Multidimensional View of Frequent Pattern Discovery



本讲内容

√ 关联规则

- u 基本概念

- u Apriori 及其改进算法

- u Apriori-based Sub-Graph Mining

Data and Knowledge Types

v Associative Pattern

- u transactional table vs relational table
- u boolean vs quantitative

v Sequential Pattern

- u A sequence : $< (ef) (ab) (df) c b >$
- u $(e,f) \rightarrow (a,b) \rightarrow c$ occur 50% of the time

v Iceburg Cube

- u table with a measures

TID	Items
10	a, c, d
20	b, c, e
30	a, b, c, e
40	b, e

TID	a	b	c	d	e
10	1	0	1	1	0
20	0	0	1	0	1
30	1	1	1	0	1
40	0	1	0	0	1

transactional=boolean

Month	City	Cust_grp	Prod	Cost	Price
Jan	Tor	Edu	Printer	500	485
Mar	Van	Edu	HD	540	520
...	...	...	...	...	...

relational with quantitative attribute

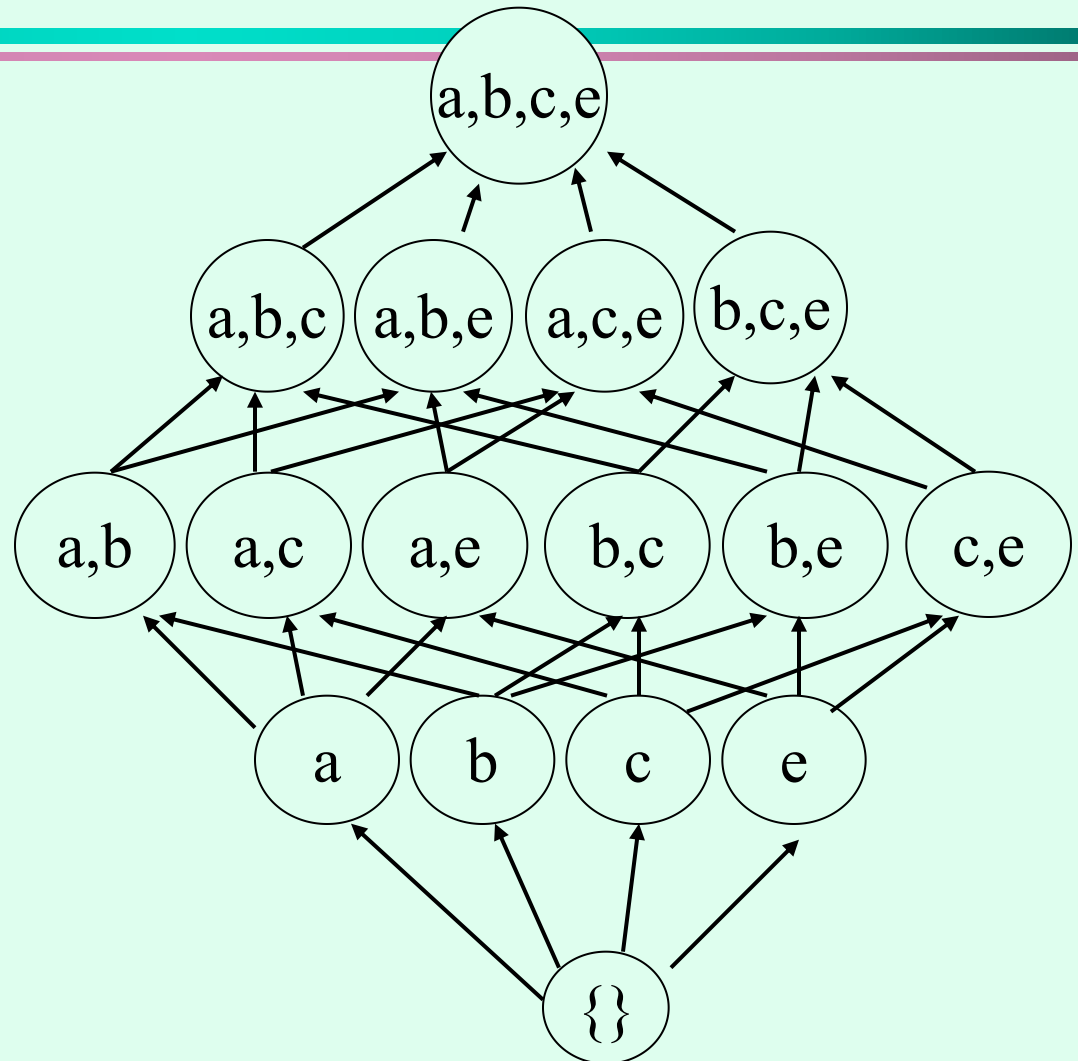
Month	City	Cust_grp	Prod	Cost (Support)
Jan	Tor	*	Printer	1040
...	...	...	...	...

cube: using other measure as support

Simulation of Lattice Transversal

The whole process of frequent pattern mining can be seen as a search in the lattice. Variety come in

- Breadth first vs Depth first
- Bottom up vs top down
- Read-based, Write-Based, Pointer-Based



Three categories of FPM Algorithms.

- ✓ **Read [AgSr94]**
 - u **Apriori-based. No write involved.**
- ✓ **Write [Zaki00, HPY00, HaPe00]**
 - u **Perform write to improve performance.**
- ✓ **Point [BeRa99, HPDW01]**
 - u **Memory based algorithm designed to point to various part of the data instead of re-writing the data in another part of the memory.**

Generalized Framework

	Read-based	Write-based	Point-based
Association Mining	Apriori[AgSr94]	FPGrowth [HaPe00]	
Sub-Graph Mining	AGM[PKDD'00] FSG[ICDM'01]	Mofa[ICDM'02]	
Sequential Pattern Discovery	GSP[AgSr96]	PrefixSpan [PHPC01]	
Iceberg Cube	Apriori[AgSr94]		BUC[BeRa99], H-cubing [HPDW01]

关联规则

- √ 关联规则是发现交易数据库中不同商品（项）之间的联系
- √ 这些规则找出顾客购买行为模式
如购买了某一商品对购买其他商品的影响
- √ 发现这样的规则可以应用于商品货架设计、存货安排以及根据购买模式对用户进行分类。

example

“尿布和啤酒”

观察发现很多顾客在买尿布的时候同时也购买啤酒，这样超市把尿布和啤酒放在一起就可以提高销售额，如果把土豆片摆在它们中间，则会同时提高这三者的销售额。

Basic Concepts

❖ 规则: $\{x_1, x_2, \dots, x_n\} \rightarrow Y$

如果顾客把商品 $x_1, x_2, \dots, x_n$ 放入购物篮中的话, 则很可能也会把商品 Y 放入其中。这个可能性(购买 $x_1, x_2, \dots, x_n$ 的前提下购买 Y)称作规则的可信度, 人们通常只对那些可信度大于某个值的规则感兴趣, 希望把这些规则找出来。当然, 有可能出现这样的情况, 某些商品是被随机放入购物篮中的, 例如: 类似这样的规则, $\{\text{牛奶, 黄油}\} \rightarrow \text{面包}$, 它的可信度非常大, 可能是因为很多人都购买面包, 象这样的规则是无用的。

❖ 可信度和最小可信度

购买 $x_1, x_2, \dots, x_n$ 的情况下购买 Y 的可能性, 条件概率

❖ 支持度和最小支持度

同时购买 $x_1, x_2, \dots, x_n$ 和 Y 的可能性

❖ 频繁项目集

满足最小支持度的项目集

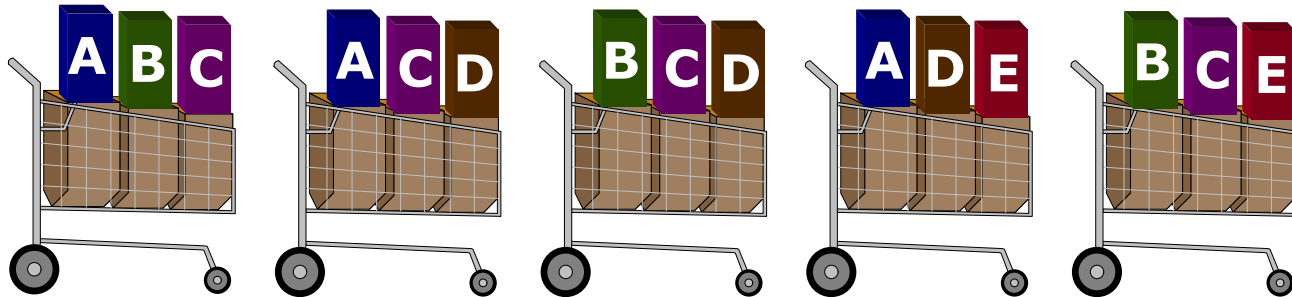
支持度:

$$\textit{support} (A \Rightarrow B) = P(A \cup B)$$

可信度:

$$\textit{confidence} (A \Rightarrow B) = P(B|A)$$

Association Rules



<u>Rule</u>	<u>Support</u>	<u>Confidence</u>
$A \Rightarrow D$	2/5	2/3
$C \Rightarrow A$	2/5	2/4
$A \Rightarrow C$	2/5	2/3
$B \ \& \ C \Rightarrow D$	1/5	1/3

因果关系

理想情况下，人们都希望从关联规则中得出这样的结论，因为购买了商品 $x_1, x_2, \dots, x_n$ ，所以一定会购买商品 Y 。但是因果关系是一个很不好确定的东西，下面举一个例子来说明对购物篮数据来说，因果关系意味着什么。如果商店降低尿布的价格而升高啤酒的价格，则会吸引尿布购买者，而这些购买尿布的人同时很可能会购买啤酒，这样商店在尿布上遭受的损失就会从啤酒中得以弥补，这是因为“尿布是因，啤酒是果”。相反，如果降低啤酒的价格而升高尿布的价格，买啤酒的人多了但买尿布的人并不会增多，商店就赔本了。

Association Rule Mining: A Road Map

- ✓ **Based on the types of values**
 - u **Boolean AR, Quantitative AR**
- ✓ **Based on the dimensions of data**
 - u **Single dimension, Multi-dimension**
- ✓ **Based on the levels of abstractions**
 - u **Single level, Multi-level**
- ✓ **Based on various extension**
 - u **Correlation analysis, Maxpatterns**

Extension of AR Mining

- ✓ **Transaction Db**

- ✓ **Relation, Data warehouse (OLAM)**

 - u **Quantitative, Multi-level, Cubegrade**

- ✓ **Constrained AR**

 - u **Constrain on Item, Interest ...**

- ❖ **Correlation analysis**

Evolution of AR Mining

❖ 1993: AIS (AR Model)

❖ 1994: Apriori (层次算法产生候选集)

- u 1995 : Partition, Hash-based, Parallel

- u 1996: Sample

- u 1997: Dynamic

❖ 2000:FP-Growth (不产生候选集)

How are association rule mined ?

两个子问题:

1. 根据用户输入的最小支持度, 寻找频繁项目集
2. 根据用户输入的最小可信度, 产生关联规则

第二个问题比较简单, 关键是解决第一个问题

Boolean AR: 算法设计的重要前提

以“市场-购物篮”为数据模型，把数据看作一个布尔矩阵，其中行=购物篮，列=商品项目：

1. 矩阵很稀疏，几乎全是0
2. 列的数目（项目数）要足够小，每列存点东西的话可以把所有的列放入内存，同时列的数目又要足够大，每个项目对存点东西的话不能全部放入内存。
3. 行的数目很多，不可能把整个矩阵放入内存，即使利用稀疏采用压缩也不行。

item TID	a	b	c	d	e
T100	1	0	1	0	0
T200	1	1	1	0	0
T300	1	0	0	1	1
T400	0	0	1	0	1
T500	1	0	1	1	1
T600	0	1	1	1	0
T700	1	0	1	0	0

怎样估计算法运行的时间？

主要考虑：

它对数据的扫描次数 数据挖掘的主要代价是从硬盘读数据，因此估算一个数据挖掘算法运行时间的最好手段是看它对同一个数据读了几次。

重要的公理

如果一个项目集 S 是频繁的（项目集 S 的出现频度大于最小支持度 s ），那么 S 的任意子集也是频繁的。

要找出频繁项目集，主要有两类算法：

1. 分层挖掘（每一层需要对数据做一次扫描。）

先找出大小为1的频繁项目集，再找出大小为2的频繁项目集，再找出大小为3的频繁项目集，等等。

我们只需要把精力放在大小为2的频繁项目集上，因为：

- u 通常，大小为2的频繁项目集就足够了。
- u 在很多数据集合里，查找大小为2的频繁项目集比较困难，进行高层挖掘比查找大小为2的频繁项目集用的时间要少。

2. 数据做一次（最多两次）扫描，找出最大频繁集（任何超集都不是频繁集的集合S）。

本讲内容

√ 关联规则

- u 基本概念

- u **Apriori** 及其改进算法

- u **Apriori-based Sub-Graph Mining**

Apriori 算法步骤1: 分层寻找频繁项目集

1. 给出minisupp, 第一次扫描数据的时候, 找出频度大于minisupp的项目, 称这集合为L1。一般来说, 一个商店卖的商品品种不会超过10万个, 所以我们假定有足够的内存来计算每个项目的出现频度。
2. 第二次扫描数据时, L1中的项目对成为大小为2的候选项目集C2, 我们希望C2不要太大, 这样的话就可以有足够的空间为每个候选对分配一个计数器来计算它的出现频度, 计数值大于minisupp的候选对构成大小为2的频繁项目集L2。
3. 第三次扫描数据时, 由L2生成大小为3的候选项目集C3, C3是这样的集合{A, B, C}, 并且{A, B}, {B, C}, {A, C}都在L2中。计算C3中每个三元组的出现频度, 大于minisupp构成L3。
4. 一直进行到集合为空时为止。Li是大小为i的频繁项目集, Ci+1是大小为i+1的集合, 这些集合的每个大小为i的子集都在Li中。

Apriori 算法步骤2: 产生关联规则

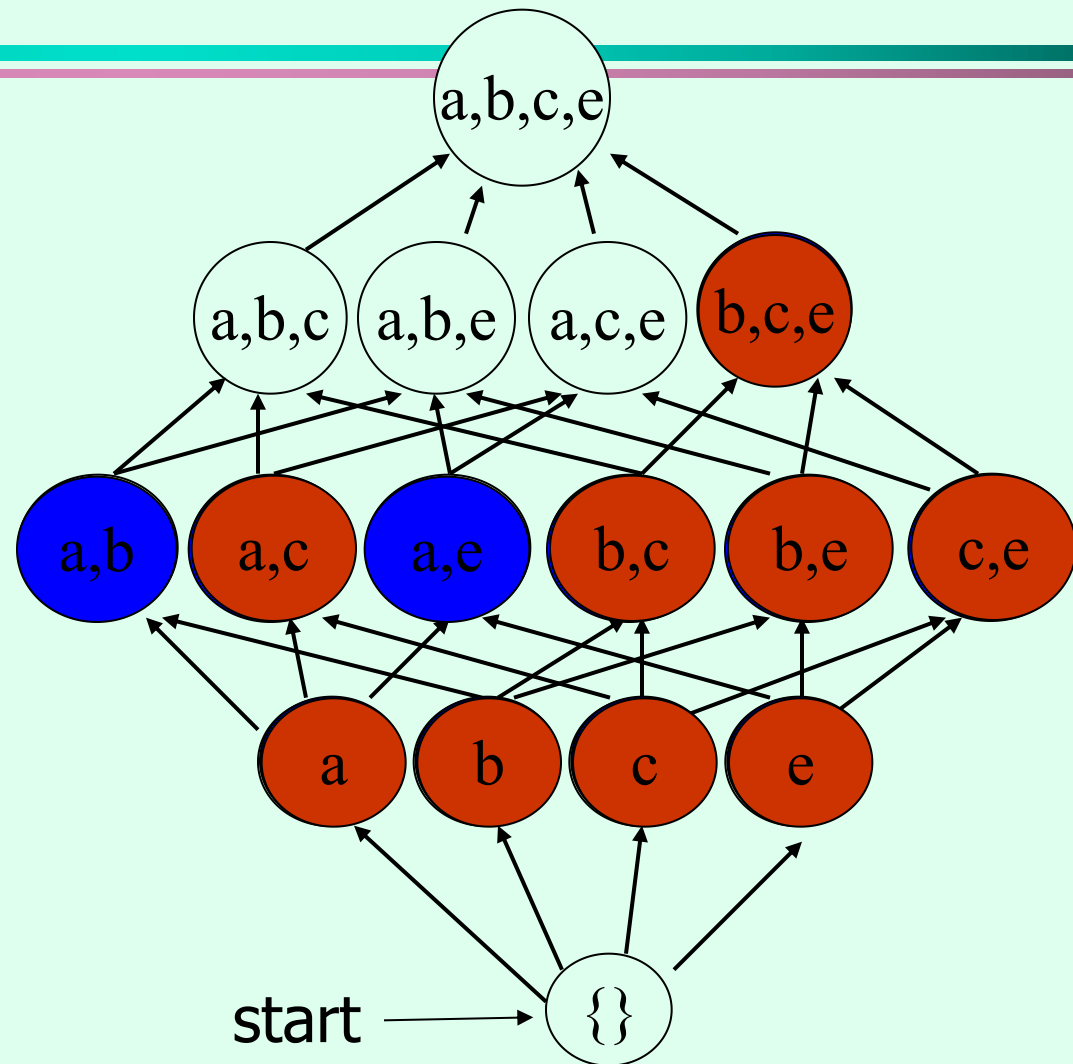
- √ 对于每个频繁项目集L，产生它的所有非空子集S
- √ 对L的每个非空子集S，如果满足如下条件：

$$\frac{\text{sup_port_count}(L)}{\text{sup_port_count}(S)} \geq \text{min_conf}$$

则输出关联规则 $S \rightarrow (L - S)$

The Apriori Algorithm

- ✓ Bottom-up, breadth first search
- ✓ Only read is perform on the databases
- ✓ Store candidates in memory to simulate the lattice search
- ✓ Iteratively follow the two steps:
 - u generate candidates
 - u count and get actual frequent items



Candidate Generation and Pruning

✓ Suppose all frequent $(k-1)$ items are in L_{k-1}

✓ **Step 1: Self-joining L_{k-1}**

insert into C_k

select $p.i_1, p.i_2, \dots, p.i_{k-1}, q.i_{k-1}$

from $L_{k-1} p, L_{k-1} q$

where $p.i_1 = q.i_1, \dots, p.i_{k-2} = q.i_{k-2}, p.i_{k-1} < q.i_{k-1}$

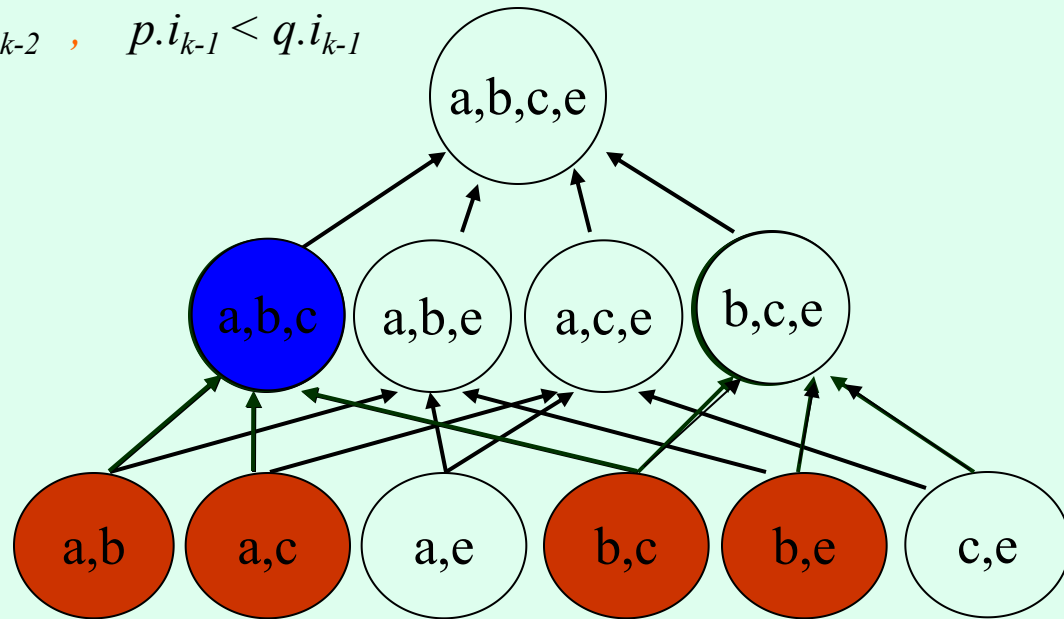
■ **Step 2: pruning**

forall **itemsets c in C_k** do

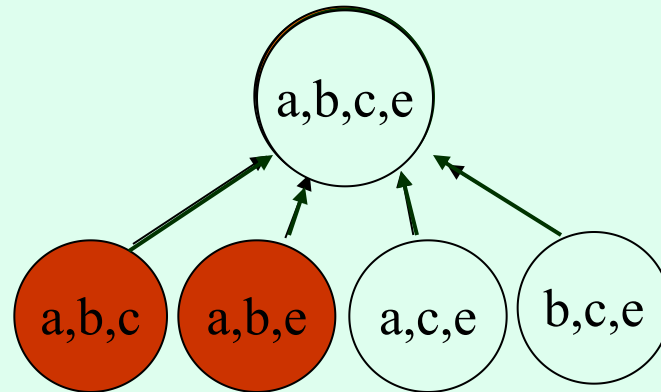
forall **$(k-1)$ -subsets s of c** do

if $(s$ is not in $L_{k-1})$ then

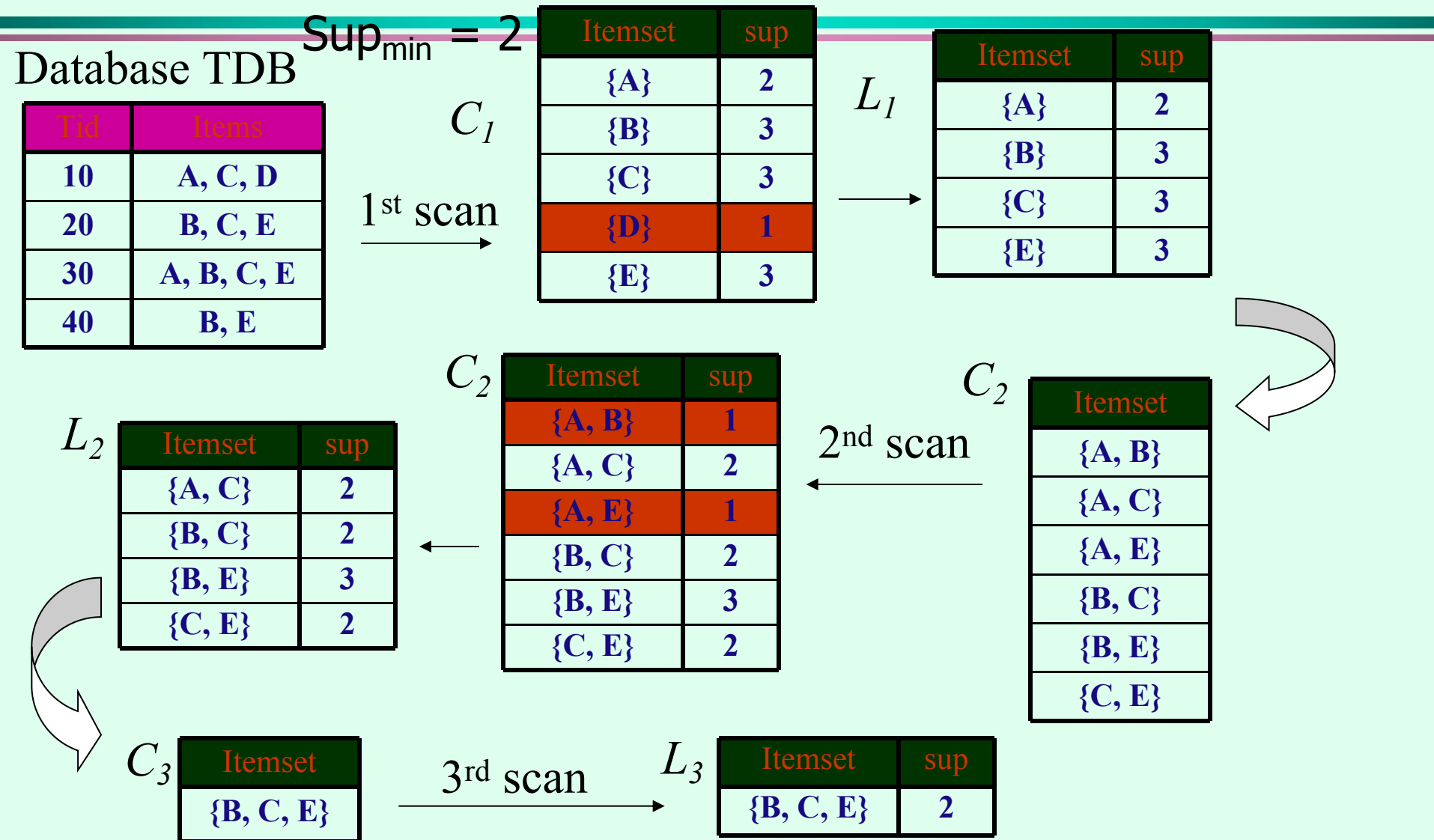
delete c from C_k



Candidate Generation and Pruning(another example)



The Apriori Algorithm—An Example



Important Details of Apriori

- ✓ How to generate candidates?
 - u Step 1: self-joining L_k
 - u Step 2: pruning
- ✓ How to count supports of candidates?
- ✓ Example of Candidate-generation
 - u $L_3 = \{abc, abd, acd, ace, bcd\}$
 - u Self-joining: $L_3 * L_3$
 - $abcd$ from abc and abd
 - $acde$ from acd and ace
 - u Pruning:
 - $acde$ is removed because ade is not in L_3
 - u $C_4 = \{abcd\}$

对A-priori算法的改进

1. 当 ≥ 2 时,减小候选集 C_i 的大小.这一点非常重要,因为即使对于找频繁的对(大小为2的频繁项目集),候选集的大小必须足够小,给每个对加一个计数器之后还可以在内存中放得下.

(基于hash的算法以及Iceberg 查询对hash算法的改进)

2. 把找 $L_1, L_2, \dots, L_n$ 的过程合并成一次(最多两次)扫描,而不是每层扫描一次.

(Sample算法, Partition算法, Dynamic算法)

3. 减小数据库的大小

任何一个事务 (一个购物蓝) , 如果它不包含 k -itemset, 则不包含 $k+1$ -itemset, 可以去掉.

(基于hash的算法采用了该技术)

基于hash的算法

利用了内存通常比项目数要大的事实,在第一次扫描数据决定L1的时候, 建立一个哈希表 (P237) , 根据该表的数据来提前去掉那些不可能是频繁的2-itemset.

在第一次开始扫描数据决定L1时, 内存中的情况如图:

Counter for item
Hash table

基于hash的算法: pass 1

1. 计算所有项目的频度
2. 对每一个记录, 将所有的 2-Itemset 哈希到哈希表的某个桶中, 桶的计数增加1
3. 扫描结束时, 将频度大于 s 的项目放入L1.
4. 扫描结束时, 找出哈希表中计数值大于 s 的桶 (在结果位图中, 相应位设为1) 。

在第二次扫描数据决定L2时，内存中的情况如图：

Frequent items
Bitmap
Counter for candidate 2-Itemset

基于hash的算法: pass 2

1. 内存中存放有大小为1的频繁项目集L1。
2. 内存中存放有位图
3. 最后，内存中还放了一个表，存放所有的候选对和它们的计数值，候选对集合C2中的每一个对 (i, j) ，必须满足下列条件：
 - u i 在L1中， j 在L1中
 - u 对 (i, j) 必须哈希到一个频繁桶中第二点正是PCY算法和直接a-priori算法之间的不同之处。PCY算法减小了第二次扫描数据时的内存需求。
4. 最后，在第二次扫描数据时，考察每一个2-Itemset，如果满足上面列出的两个条件，则在内存中增加它的计数值，如果第一次出现的话创建一个入口点。

An example

Database D

TID	Items
100	A C D
200	B C E
300	A B C E
400	B E

Making a hash table

100	{A C}, {A D}, {C D}
200	{B C}, {B E}, {C E}
300	{A B}, {A C}, {A E}, {B C}, {B E}, {C E}
400	{B E}

$$h(\{x y\}) = ((\text{order of } x) * 10 + (\text{order of } y)) \bmod 7;$$

Items	Order
A	1
B	2
C	3
D	4
E	5

On the fly

C_1
 $\begin{Bmatrix} A \\ B \\ C \\ D \\ E \end{Bmatrix}$

count
 2
 3
 3
 1
 3

L_1
 $\begin{Bmatrix} A \\ B \\ C \\ E \end{Bmatrix}$

minimum support, $s = 2$

$\begin{Bmatrix} C E \\ C E \\ A D \end{Bmatrix}$	$\{A E\}$	$\begin{Bmatrix} B C \\ B C \end{Bmatrix}$		$\begin{Bmatrix} B E \\ B E \\ B E \end{Bmatrix}$	$\{A B\}$	$\begin{Bmatrix} A C \\ C D \\ A C \end{Bmatrix}$
3	1	2	0	3	1	3
0	1	2	3	4	5	6

Hash table H_2
 Hash address

The number of items hashed to bucket 2

Generating C_2

in a bucket with the itemset

$L_1 \times L_1$

$\begin{Bmatrix} A B \\ A C \\ A E \\ B C \\ B E \\ C E \end{Bmatrix}$

1
 3
 1
 2
 3
 3

C_2

$\begin{Bmatrix} A C \\ B C \\ B E \\ C E \end{Bmatrix}$

什么时候Hash算法比Apriori 算法优越？

当C2很大（C2和他们的计数器在内存中放不下），而Hash算法产生的频繁桶却很少（大大地减小了C2的大小，使之能全部装入内存,从而不需要再扫描数据）时，Hash算法比Apriori算法优越。

partition算法 (two passes)

将数据划分成多个子集, 分别读入内存, 找出局部候选集(pass 1)

再次扫描数据, 找出所有的频繁集.(pass 2)

关键点: 一个集合如果在任一个子集中都不频繁, 则它在全局数据中也不频繁

Sample算法(one or two passes)

随机取内存大小的一个数据的样本，在内存中运行一个层次算法（不用负担 I/O），希望从这个样本数据能求出真正的频繁集

- u 按比例伸缩最小支持率，如果样本数据是整个数据的1%，则应该用s%作为新的最小支持率
- u 完整地扫描数据一次，对由样本数据得出的频繁项目集进行验证，但是那些在全局数据中频繁但在样本数据中不频繁的集合被丢掉了
- u 为了减小false negative，可以将样本数据的最小支持率降低一点，这样在全面扫描数据的时候就可以找到更多的候选集。但注意：这样做的后果可能会产生太多的候选集而使内存装不下。

Bottleneck of Frequent-pattern Mining

- ✓ Multiple database scans are **costly**
- ✓ Mining long patterns needs many passes of scanning and generates lots of candidates
 - u To find frequent itemset $i_1 i_2 \dots i_{100}$
 - # of scans: **100**
 - # of Candidates: $\binom{100}{1} + \binom{100}{2} + \dots + \binom{100}{100} = 2^{100} - 1 = \mathbf{1.27 * 10^{30} !}$
- ✓ **Bottleneck: candidate-generation-and-test**
- ✓ Can we avoid candidate generation?

So far, we covered

	Read-based	Write-based	Point-based
Association Mining	Apriori[AgSr94]	FPGrowth [HaPe00]	
Sub-Graph Mining	AGM[PKDD'00] FSG[ICDM'01]	Mofa[ICDM'02]	
Sequential Pattern Discovery	GSP[AgSr96]	PrefixSpan [PHPC01]	
Iceberg Cube	Apriori[AgSr94]		BUC[BeRa99], H-cubing [HPDW01]

本讲内容

√ 关联规则

- u 基本概念

- u Apriori 及其改进算法

- u Apriori-based Sub-Graph Mining

Graph Mining

✓ **Graphs are ubiquitous**

- u **Web databases, XML databases**
- u **Cheminformatics (chemical compound)**
- u **Bioinformactics (protein structure, pathway)**
- u **Workflow analysis**
- u **Social network analysis**

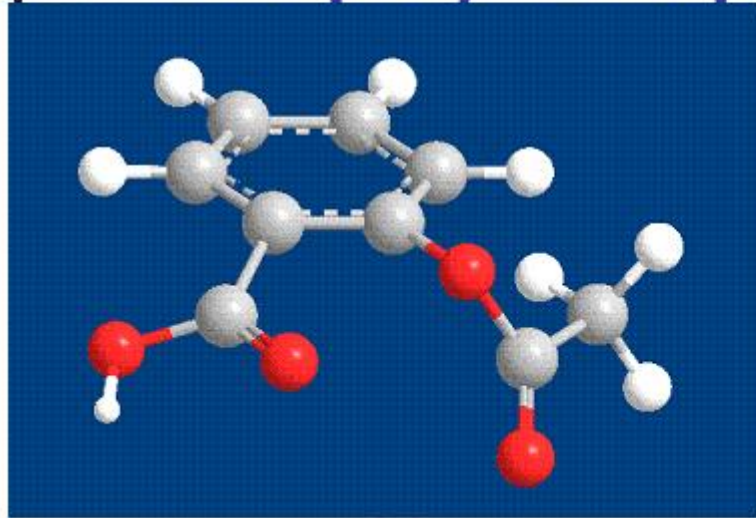
✓ **Graph is a general model**

- u **Trees, lattices, sequences, and items are degenerated graphs**
- u **Graph algorithms: many have high complexity**

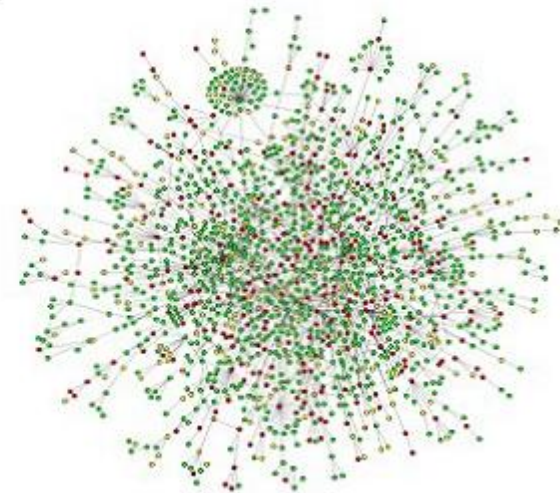
✓ **Diversity of graphs**

- u **Directed vs. undirected, labeled vs. unlabeled (edges and vertices), angles/geometry, weighted edges**

Graph, Graph, Everywhere

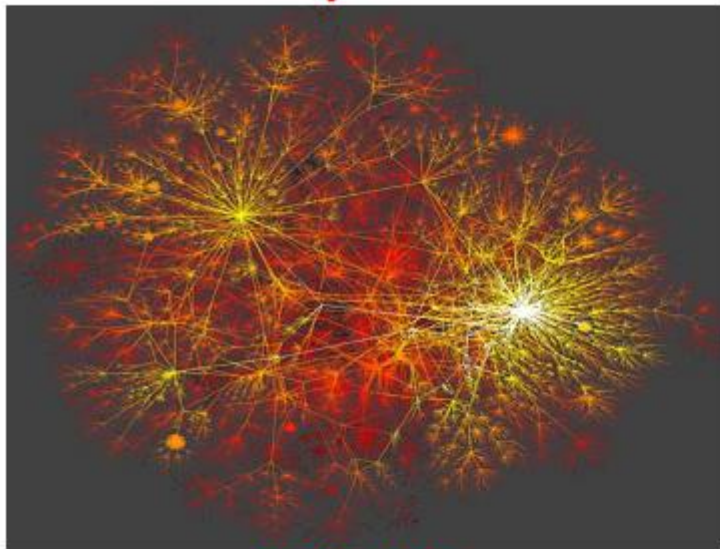


Aspirin

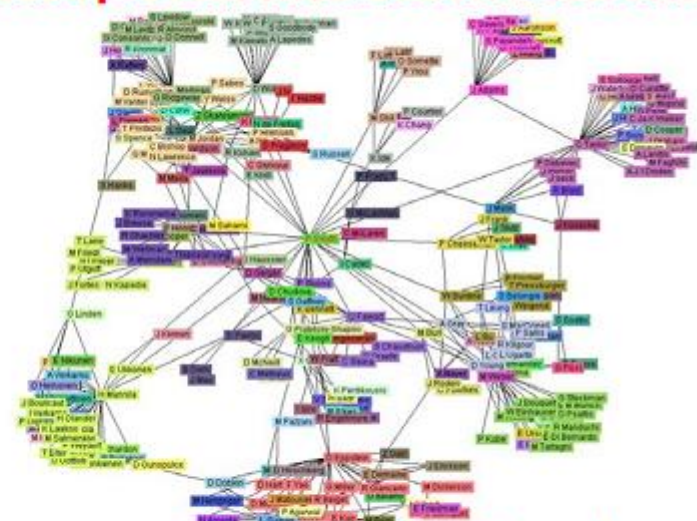


Yeast protein interaction network

from H. Jeong et al Nature 411, 41 (2001)



Internet



Co-author network

Graph Mining: Big Picture

+ Graph Level

-Patterns

-Laws

-Generators

+ Subgraph Level

- Community

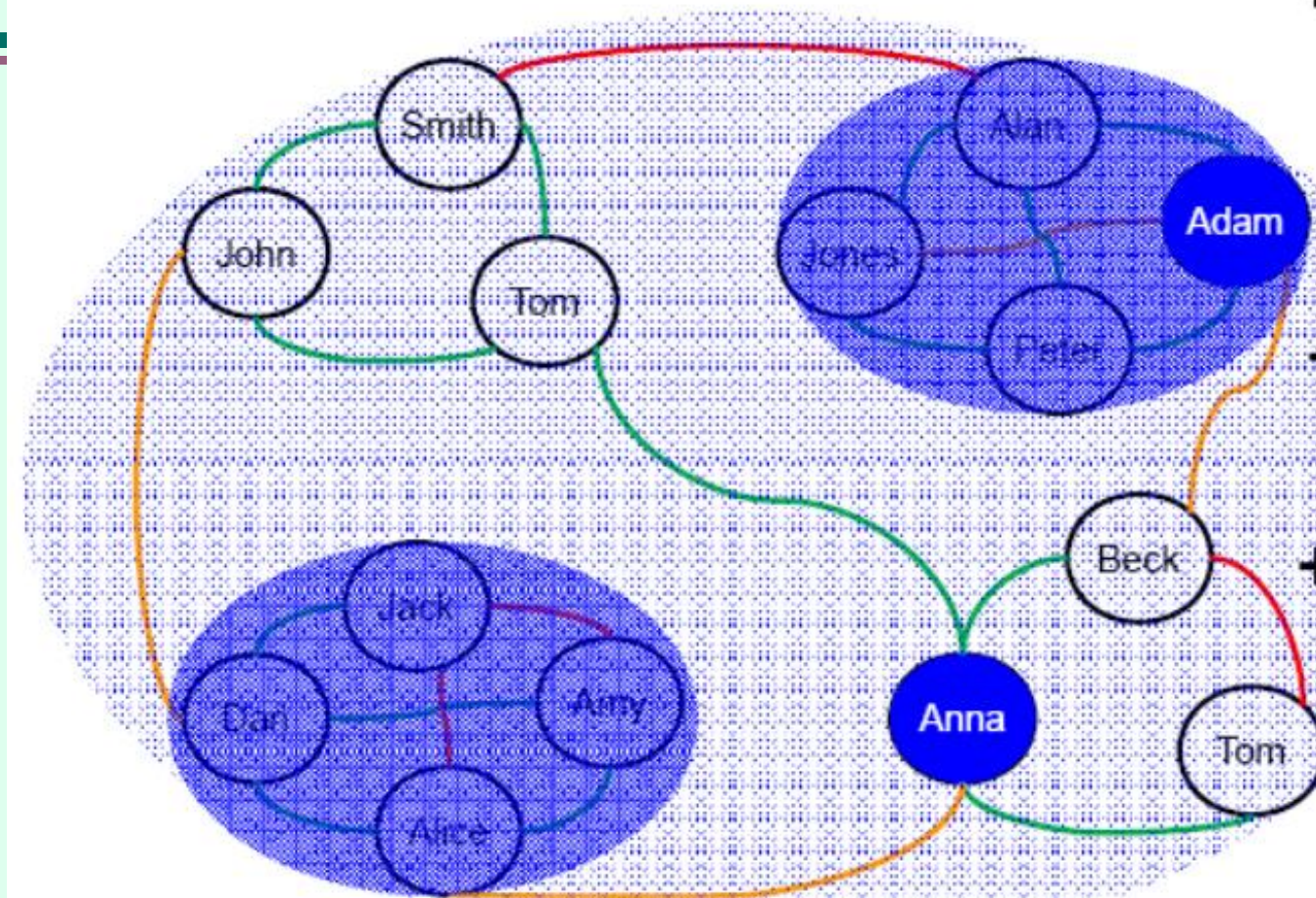
+ Node Level

-Association

-Correlation

-Causality

-Proximity

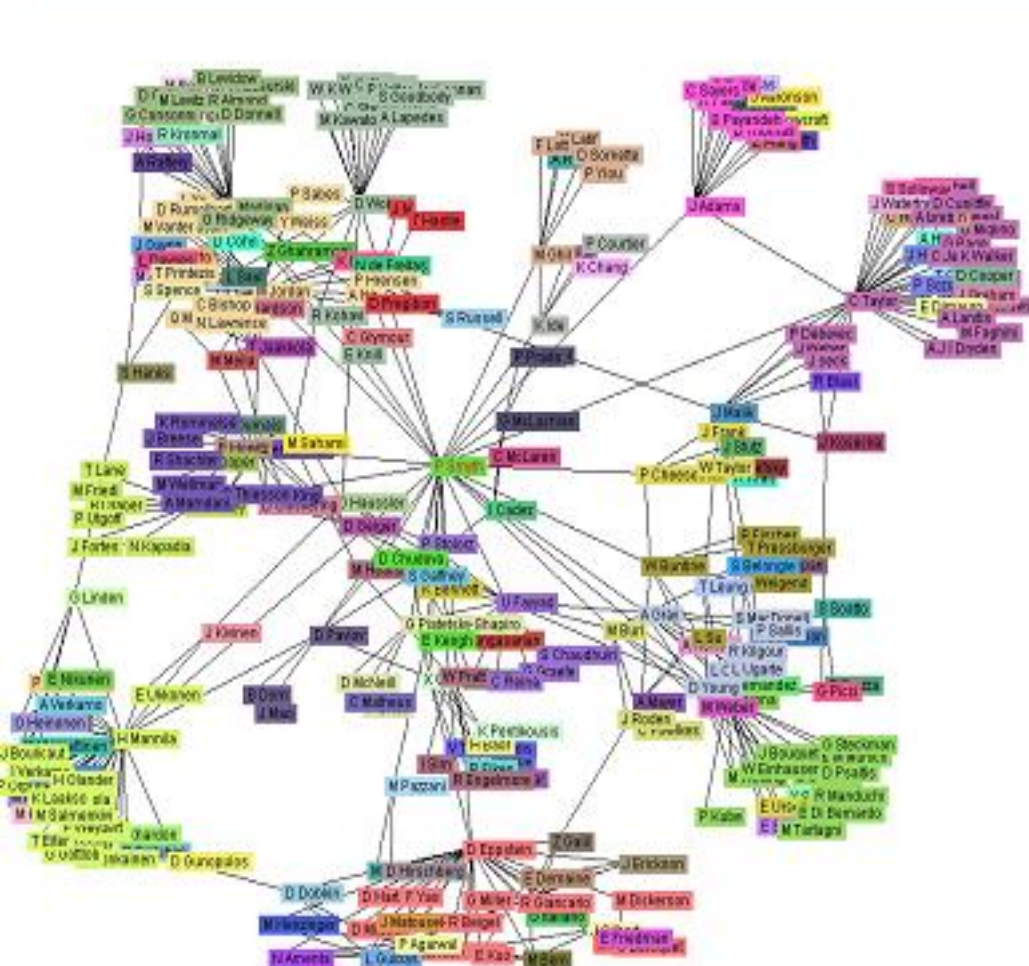


Cell Phone

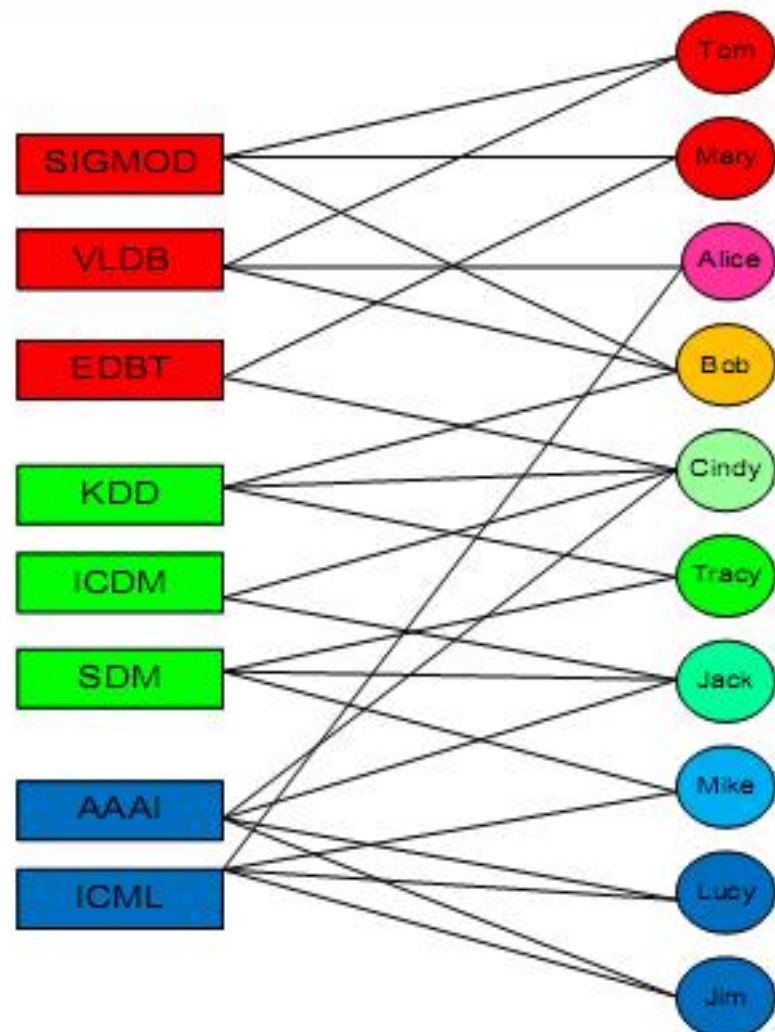
Message

E-Mail

Homogeneous vs. Heterogeneous Networks

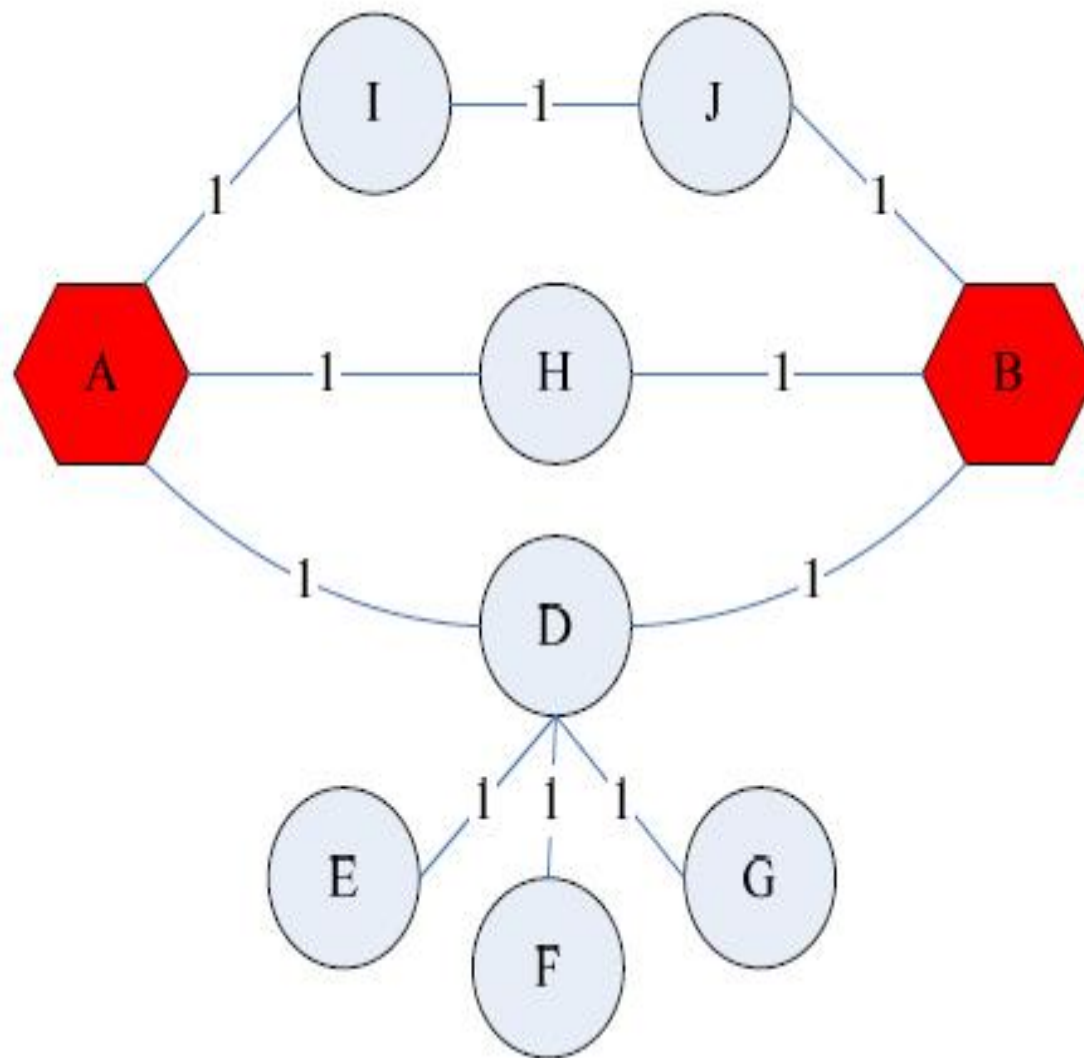


Co-author (Homogeneous) Network



Venue-Author (Heterog.) Network

Proximity on Graph: What?

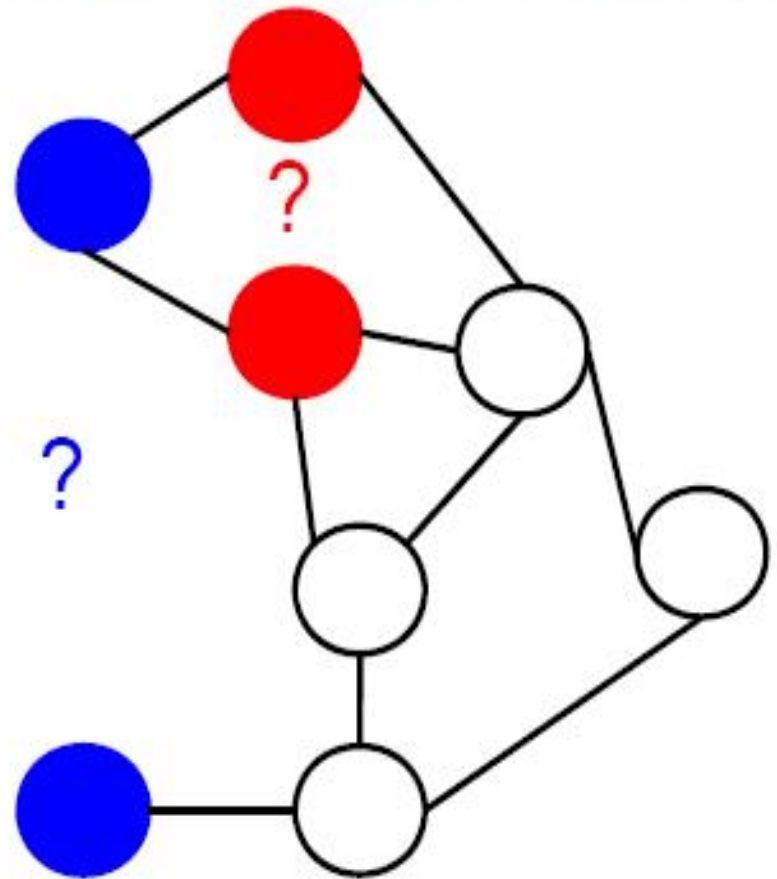
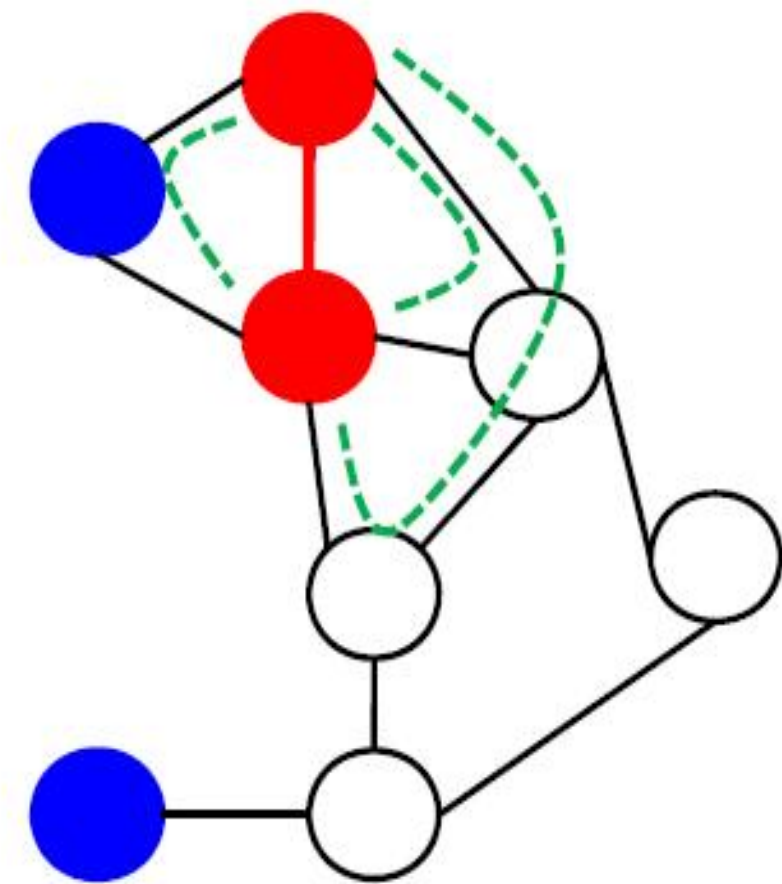


a.k.a Relevance, Closeness, 'Similarity'...

Proximity on Graphs: Why?

- **A1**: Link Prediction [Liben-Nowell+], [Tong+]
- **A2**: Neighborhood Search [Sun+]
- **A3**: Image Captioning [Pan+]
- **A4**: Conn. Subgraph [Faloutsos+], [Tong+], [Koren+]
- **A5**: Best Effort Pattern match [Tong+]
- **A6**: Proximity Tracking [Tong+]
- **A7**: Mining Time [Chakrabarti+]
- **A8**: Interactive Querying and Summarization [Tong+]
- Many more...

A1: Link Prediction

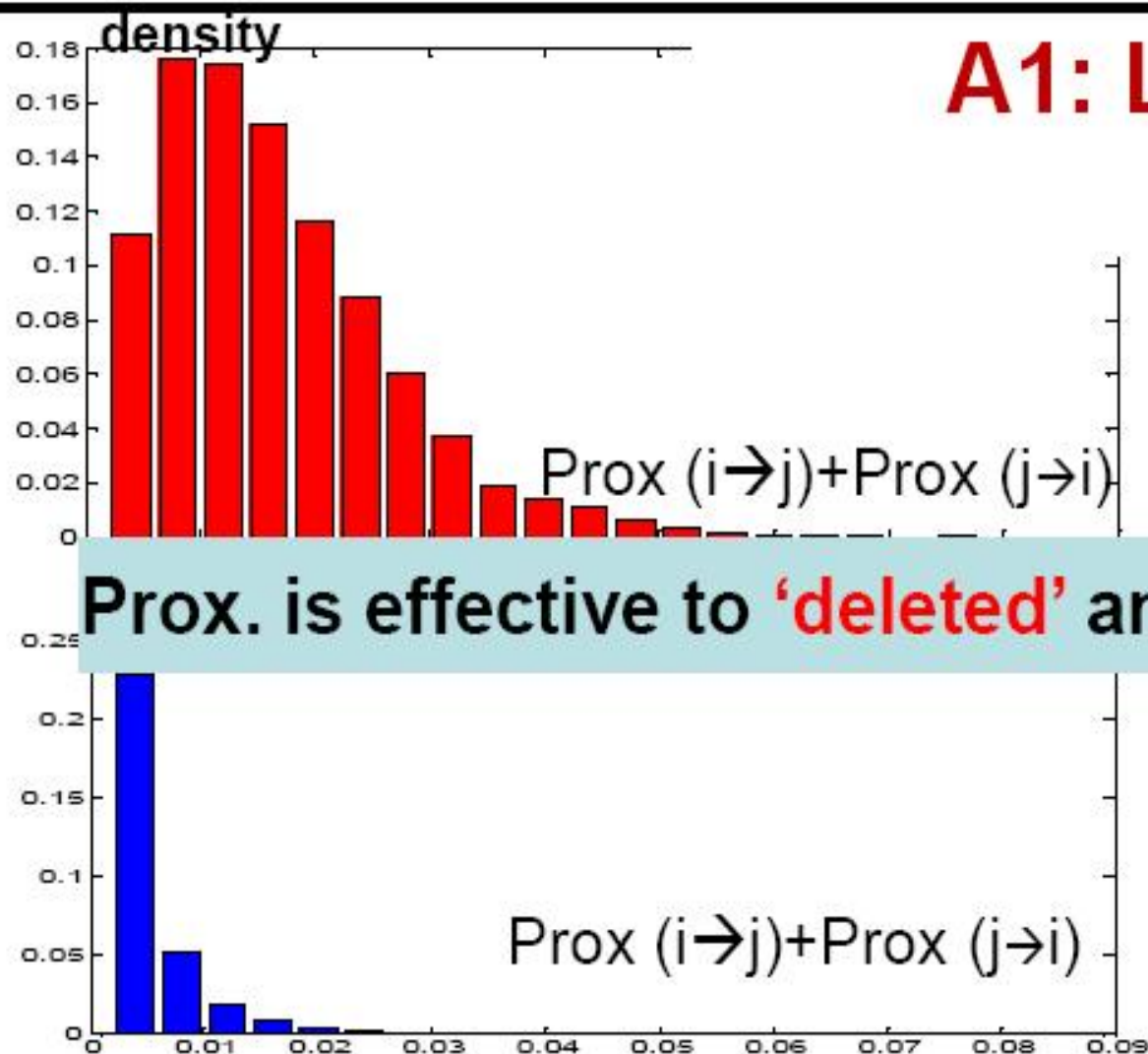


Footnote:

- Red pair: ``deleted'';
- Blue pair: ``absent''

Prox (deleted) >> Prox (absent) !

A1: Link Prediction-existence



Prox. Hist. for a set of **deleted** links

Prox. is effective to '**deleted**' and **absent** edges!

Prox. Hist. for a set of **absent** links

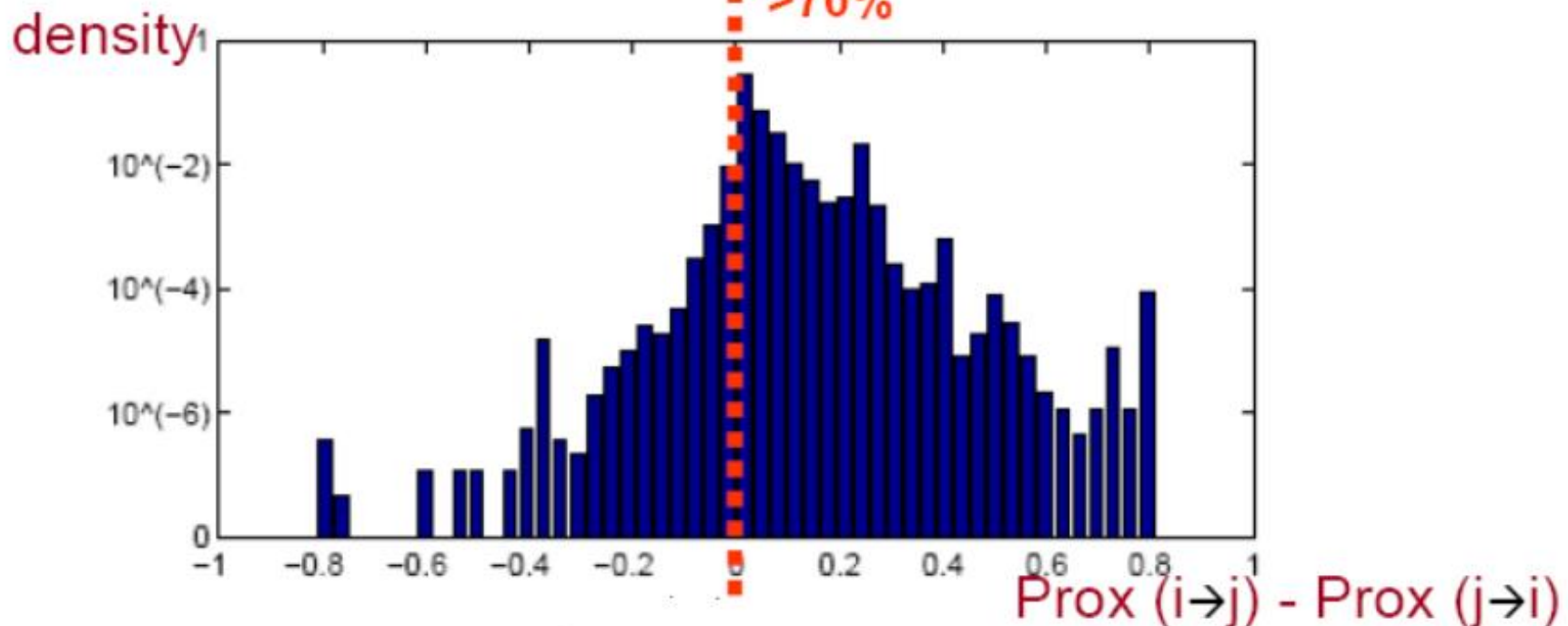
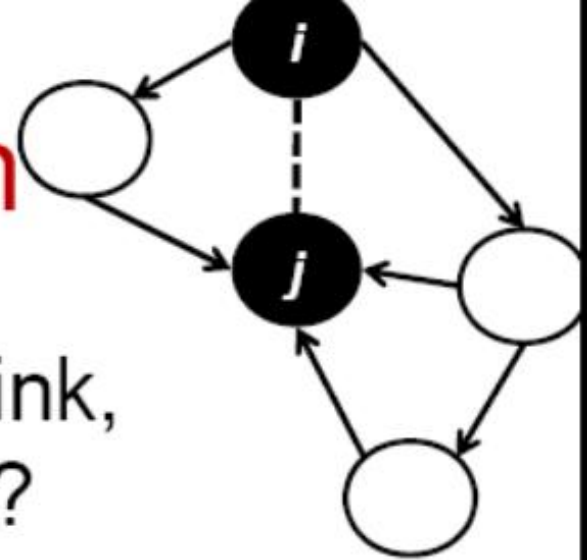
Q: How to predict the existence of the link?

A: **Proximity!** [Liben-Nowell + 2003]

Link Prediction: direction

[Tong+ KDD 07 a]

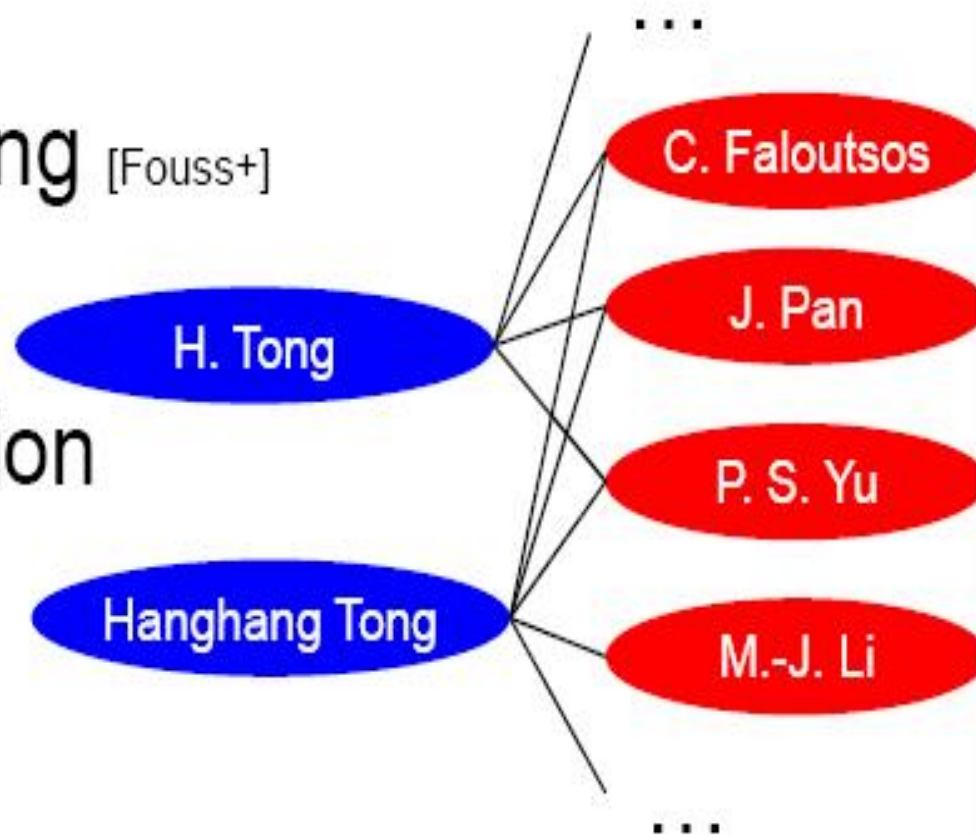
- Q: Given the existence of the link, what is the direction of the link?
- A: Compare $\text{prox}(i \rightarrow j)$ and $\text{prox}(j \rightarrow i)$



Beyond Link Prediction

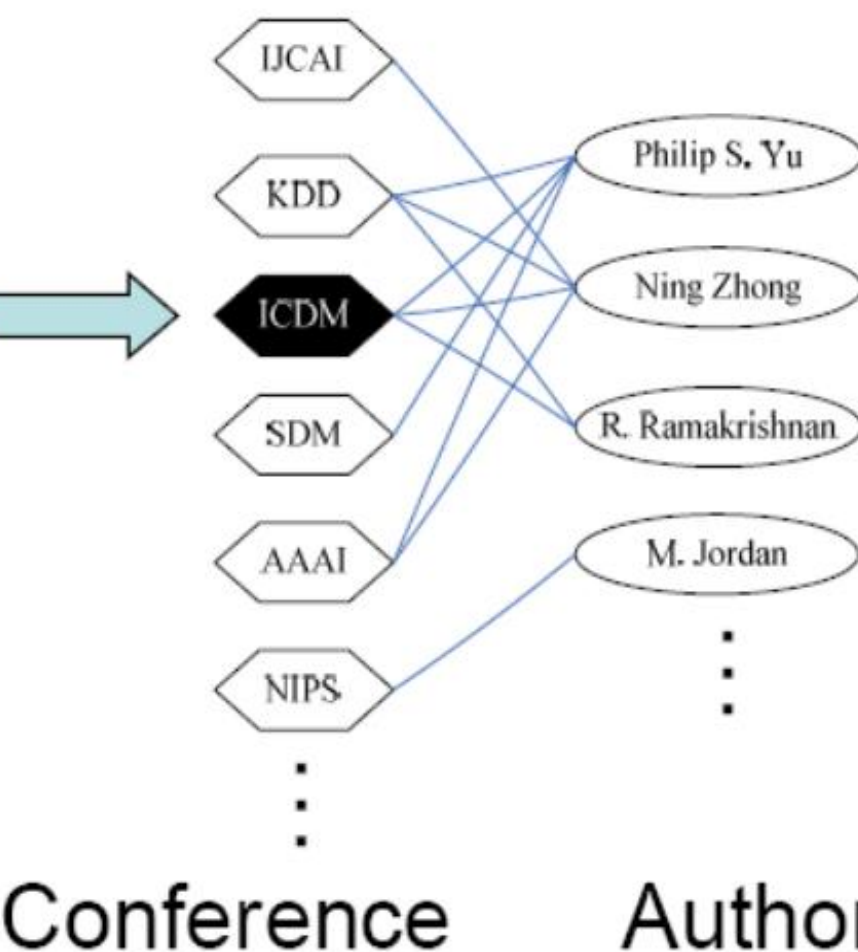
- Collaborative Filtering [Fouss+]

- Name Disambiguation
 - [Minkov+ SIGIR 06]



- Anomaly Nodes/Edges
 - 'a' is abnormal if the neighborhood of 'a' is so different
 - [Sun+ ICDM 2005]

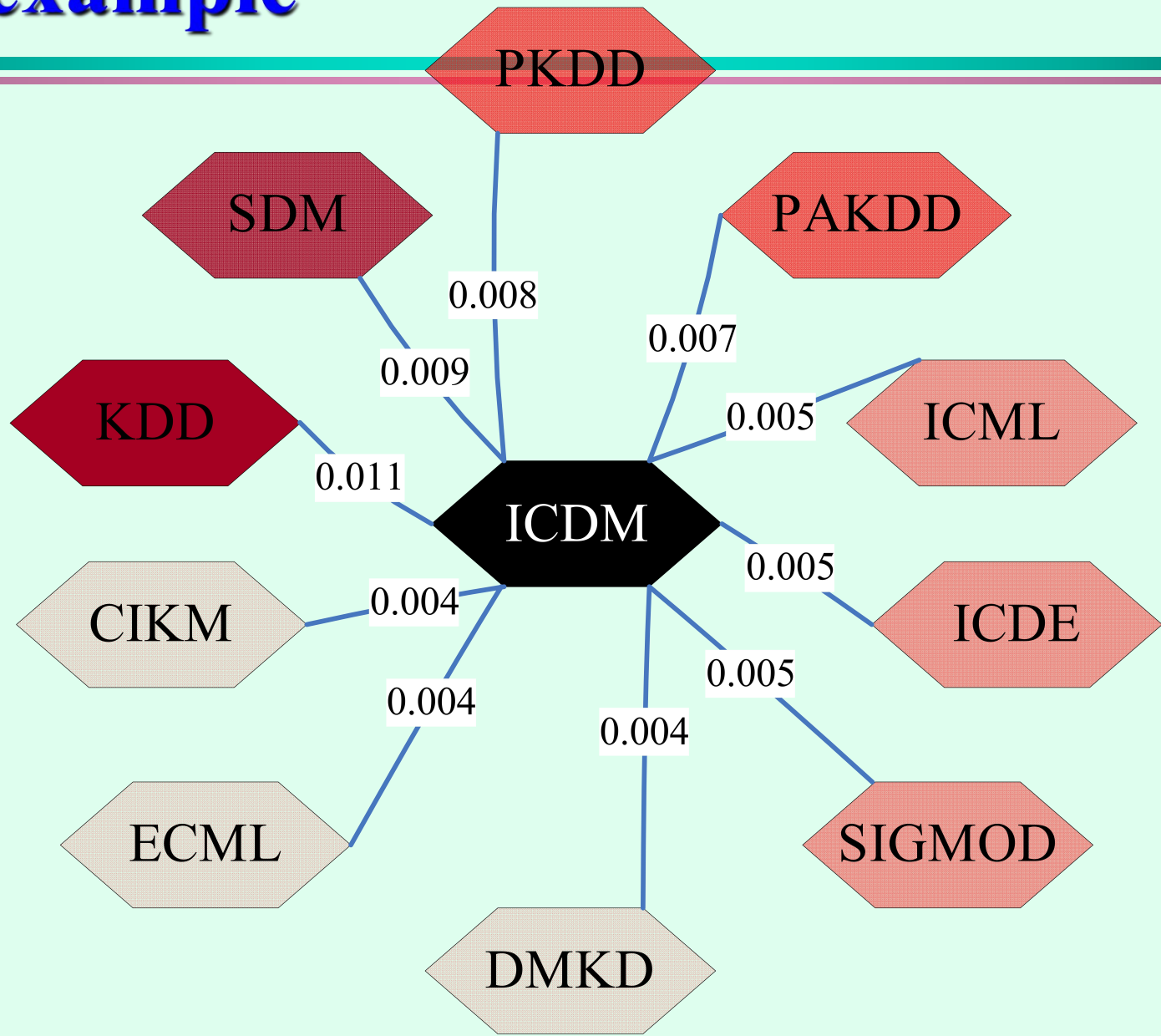
A2: Neighborhood Search on graphs



Q: what is most related conference to ICDM?

A: Proximity! [Sun+ ICDM2005]

NF: example



Automatic Image Caption

v Q

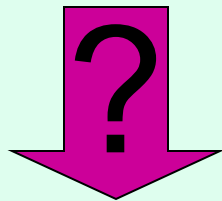


{Sea Sun Sky Wave}

...



{Cat Forest Grass Tiger}

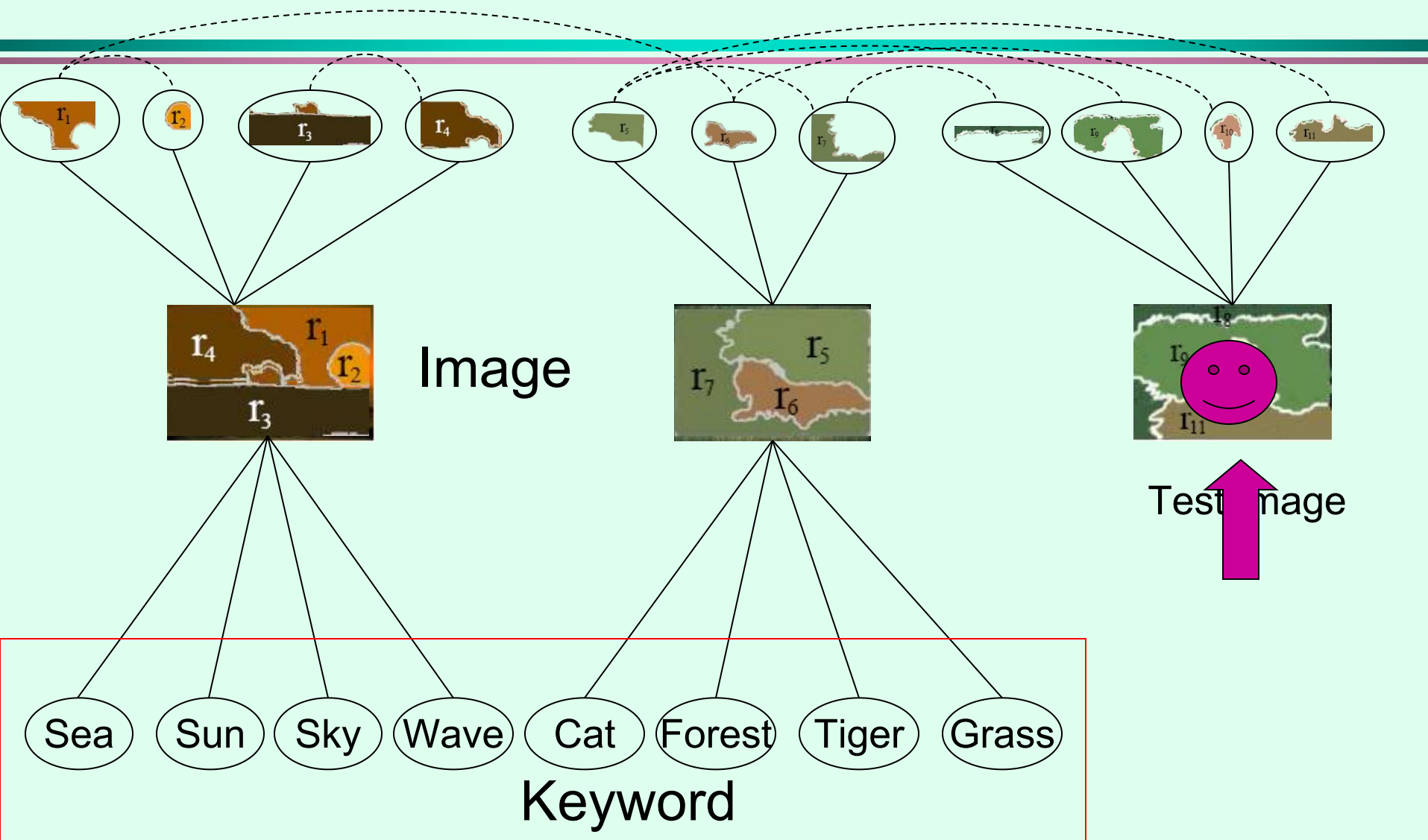


{?, ?, ?,}

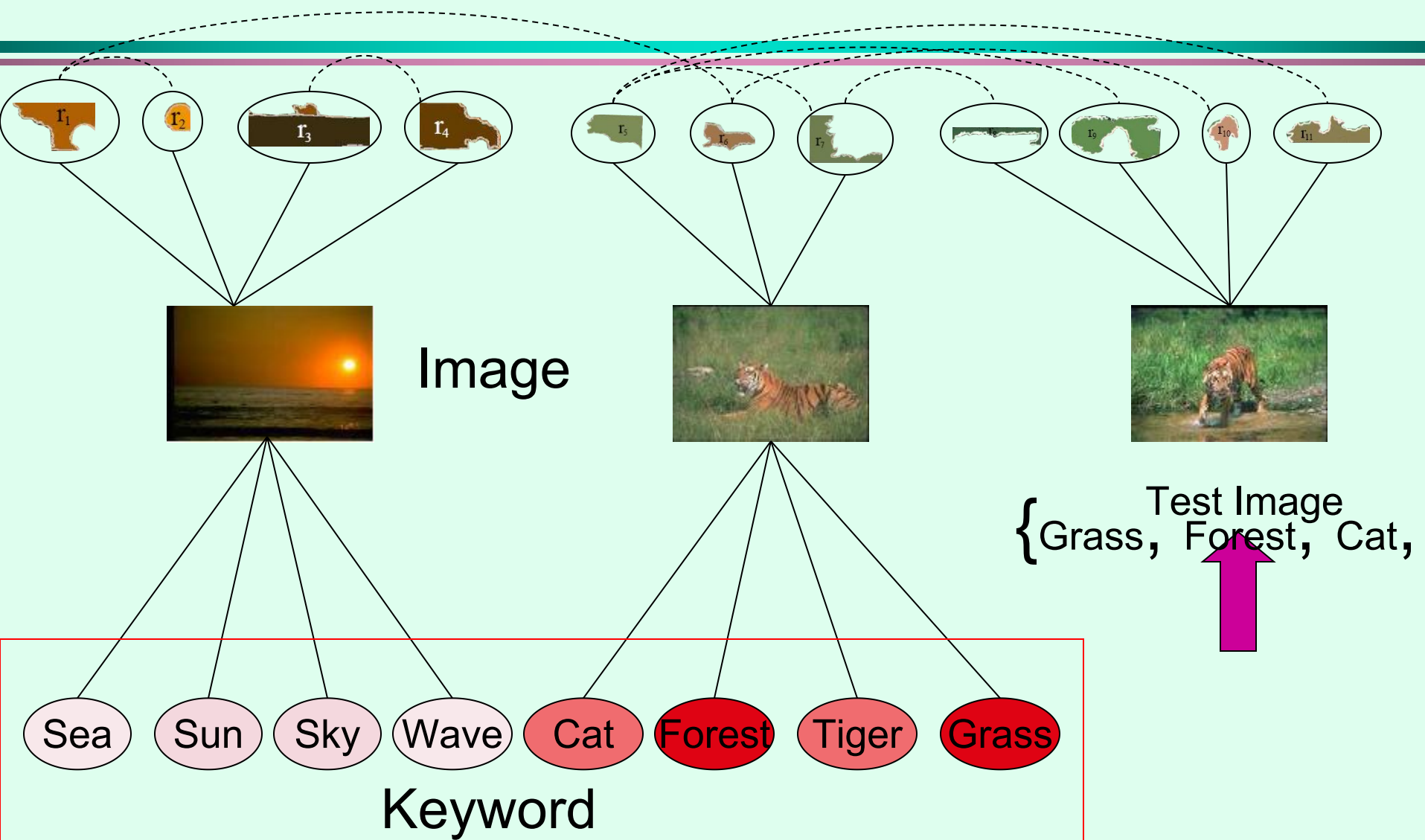
A: RWR!

[Pan KDD2004]

Region

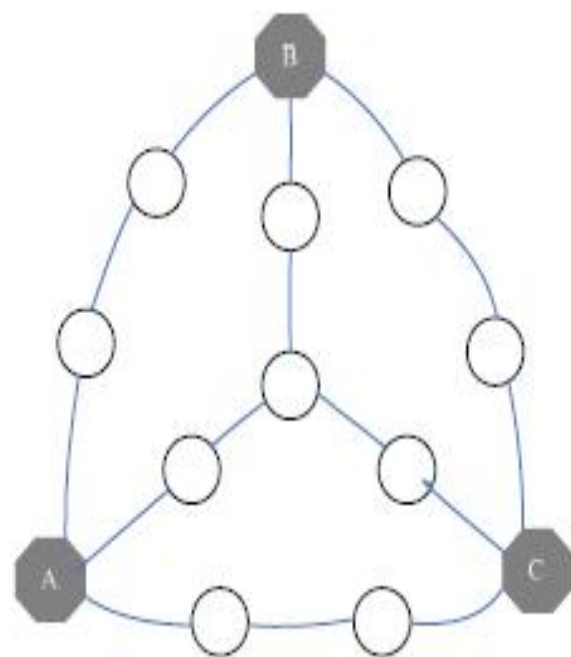


Region

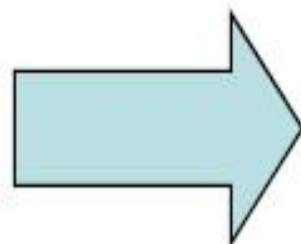


A4: Center-Piece Subgraph(CePS)

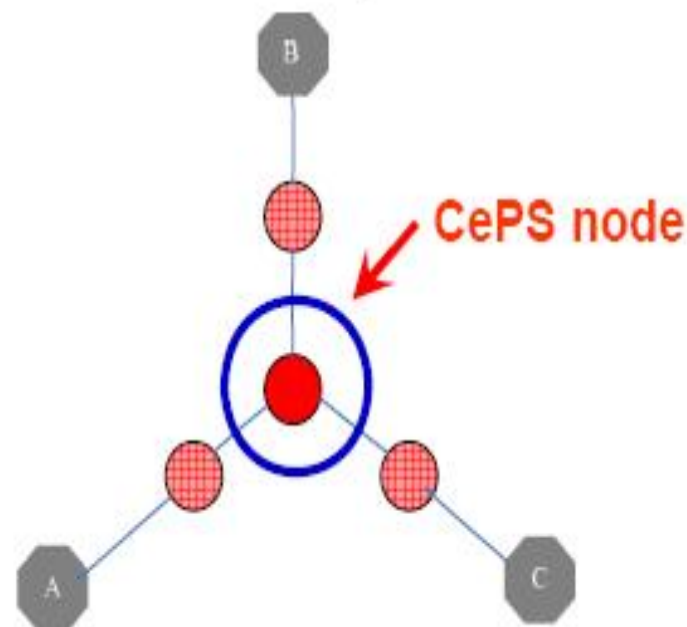
Input



Original Graph



Output



CePS

Q: How to find hub for the black nodes?

A: Proximity! [Tong+ KDD 2006]

CePS: Example

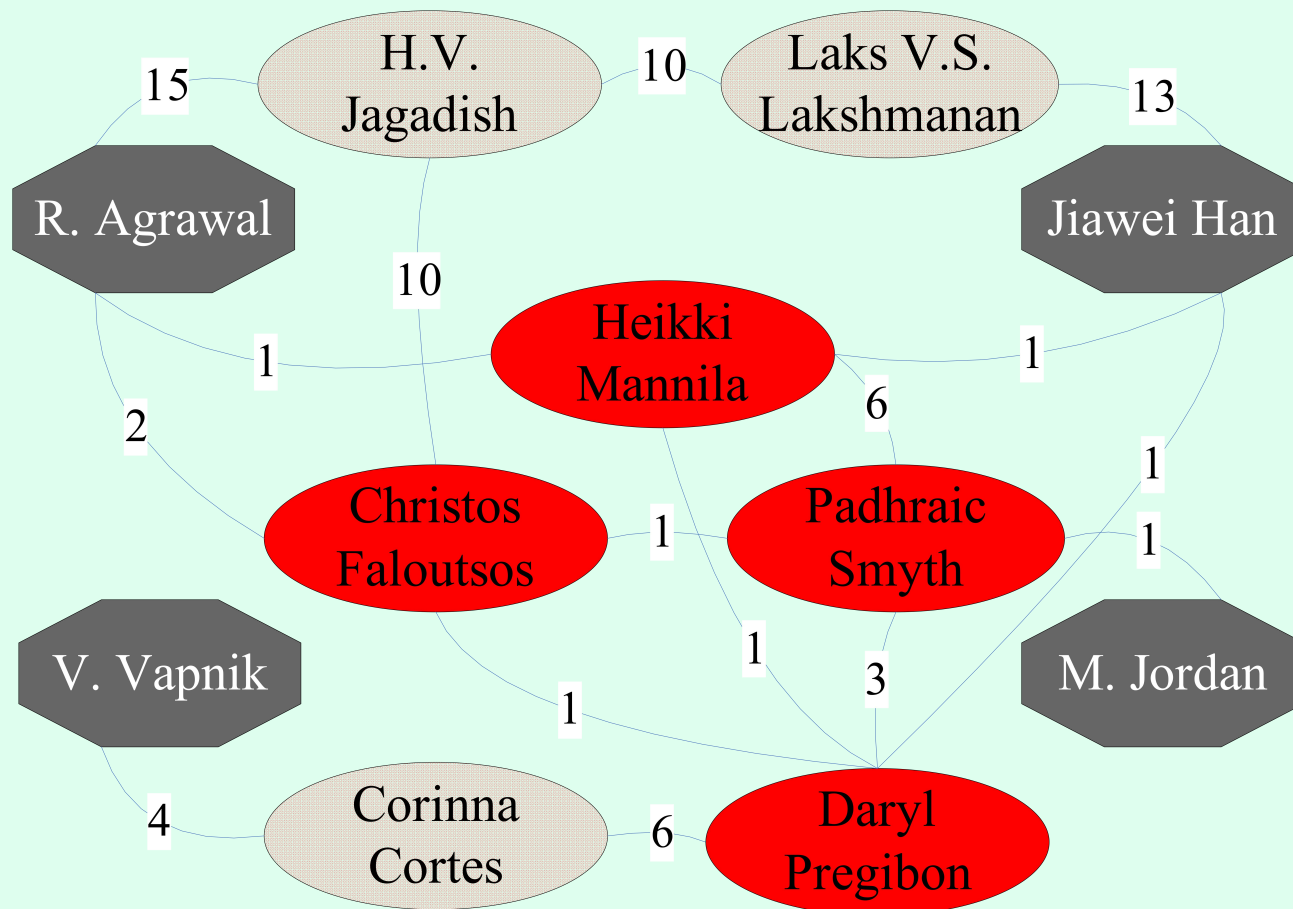
R. Agrawal

Jiawei Han

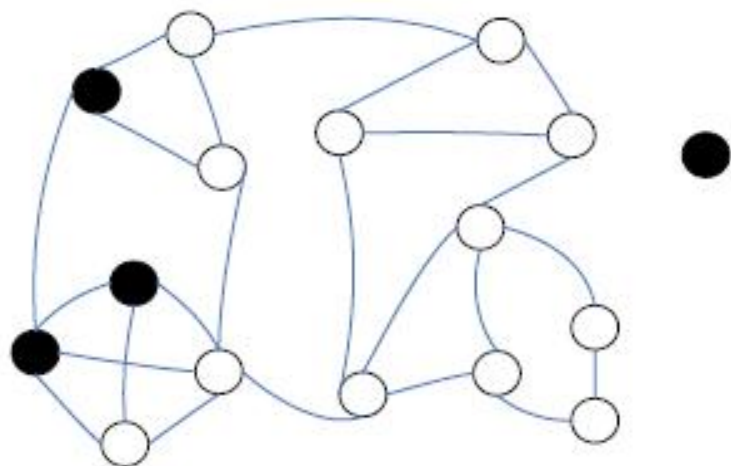
V. Vapnik

M. Jordan

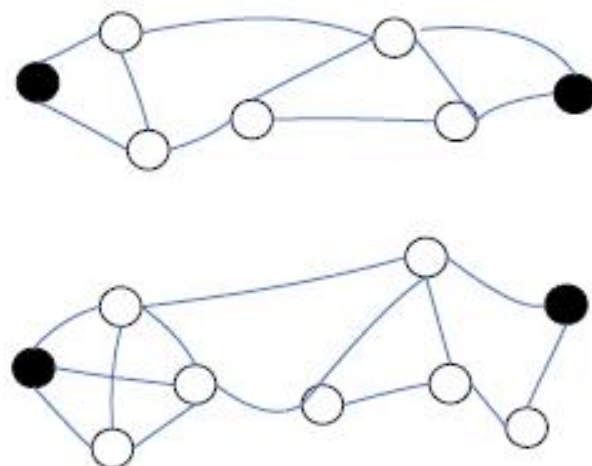
CePS: Example



K_SoftAnd: Relaxation of AND



Noise



Disconnected
Communities

Asking AND query? → No Answer!

CePS: 2-SoftAND

R. Agrawal

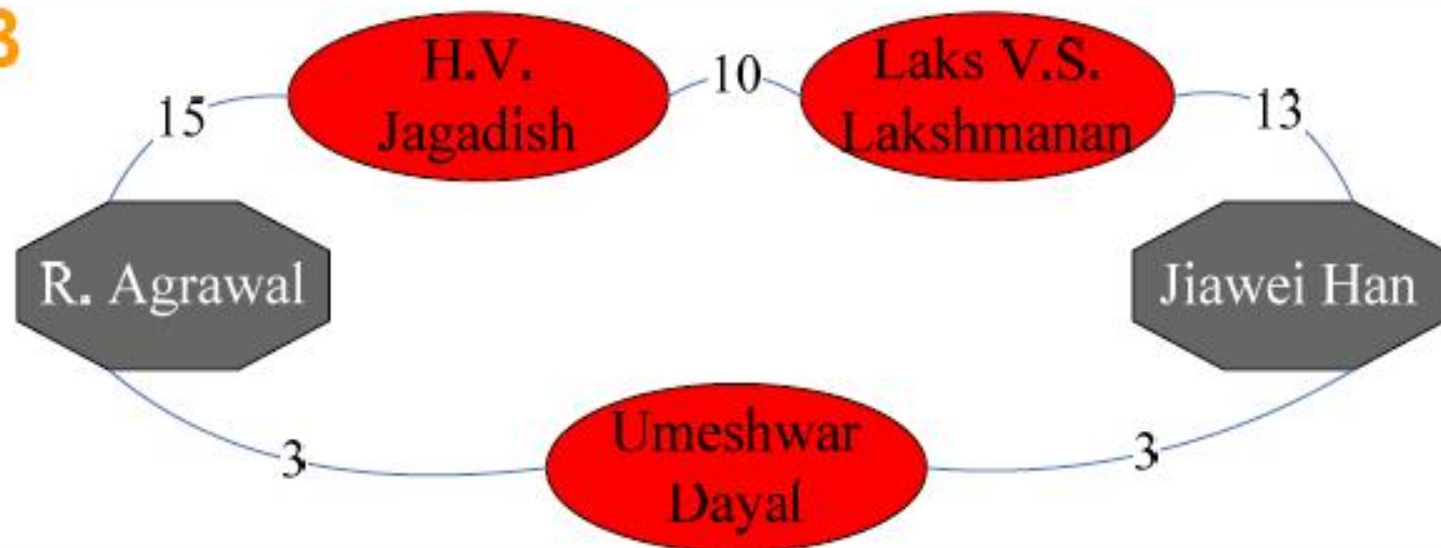
Jiawei Han

V. Vapnik

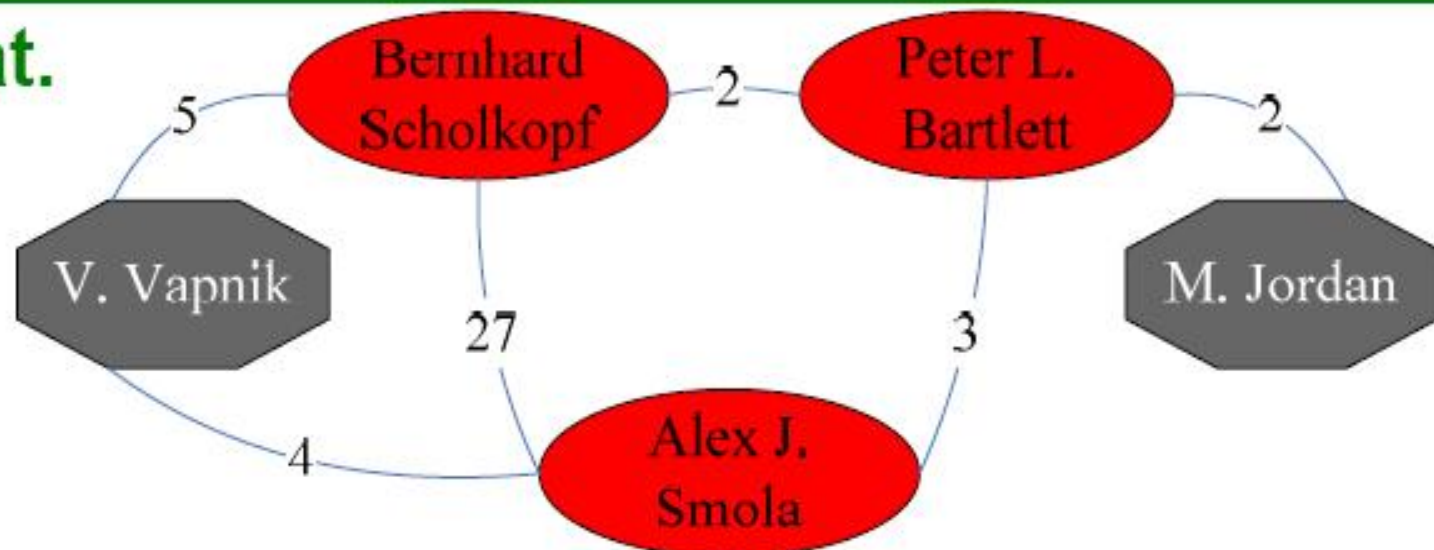
M. Jordan

CePS: 2 SoftAND

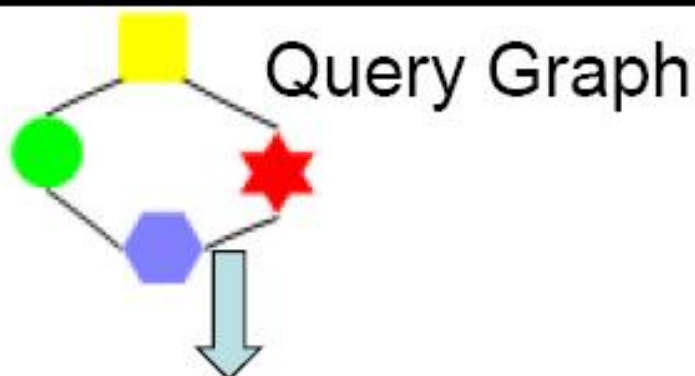
DB



Stat.



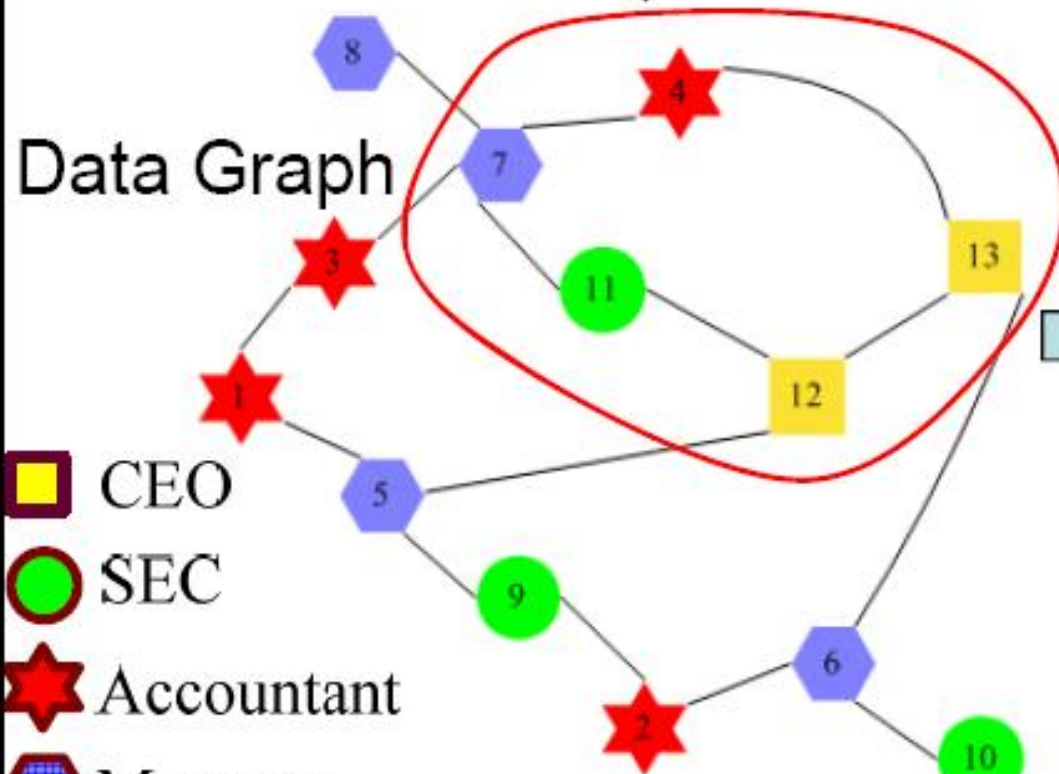
Input



Output

A5: Best-Effort
Pattern Match

Data Graph



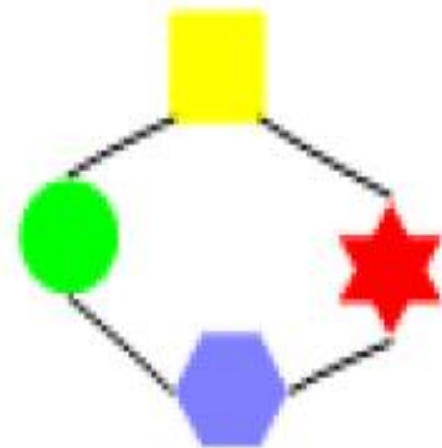
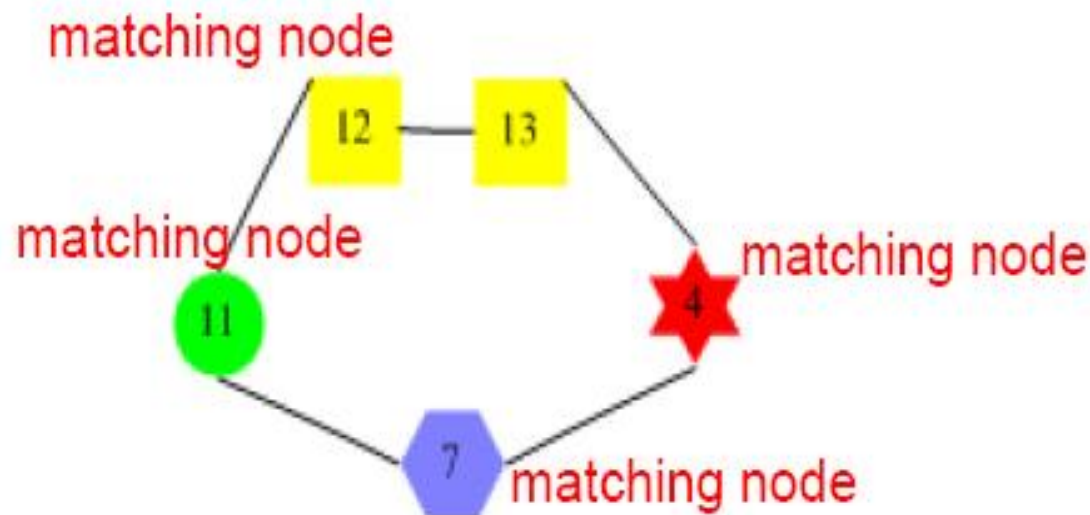
Matching Subgraph

Q: How to find matching subgraph?

A: Proximity! [Tong+ KDD 2007 b]

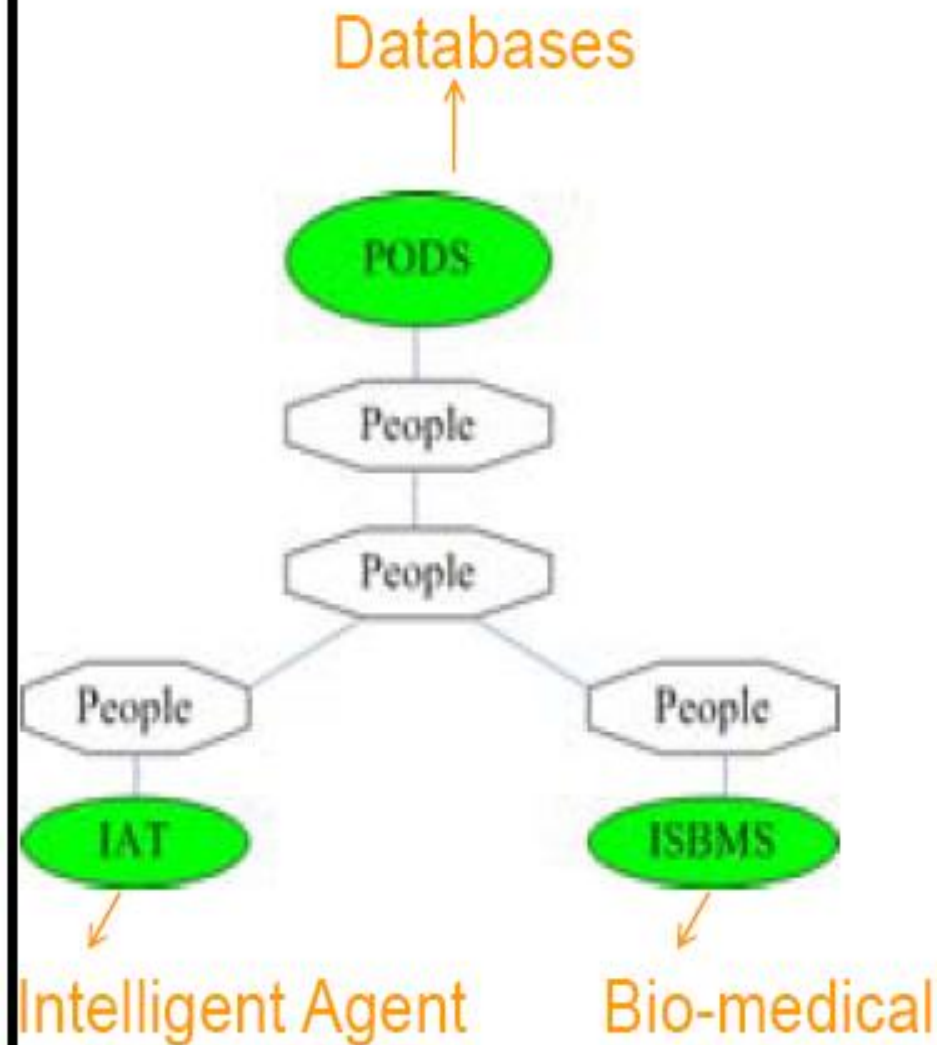
G-Ray: How to?

details

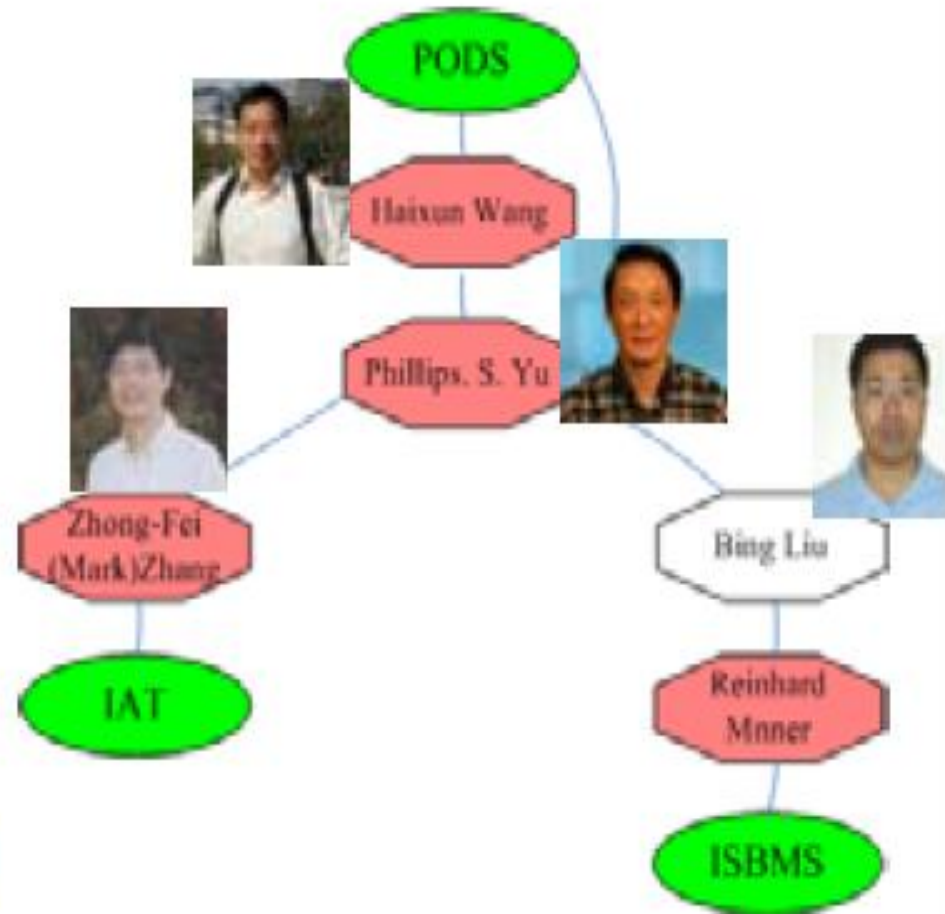


$$\begin{aligned} \text{Goodness} = & \text{Prox}(12, 4) \times \text{Prox}(4, 12) \times \\ & \text{Prox}(7, 4) \times \text{Prox}(4, 7) \times \\ & \text{Prox}(11, 7) \times \text{Prox}(7, 11) \times \\ & \text{Prox}(12, 11) \times \text{Prox}(11, 12) \end{aligned}$$

Effectiveness: star-query



Query



Result

Frequent Graph Pattern Mining

✓ *Frequent* subgraphs

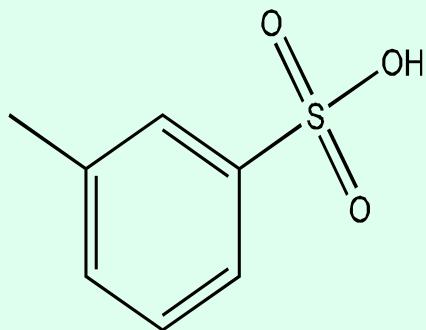
- u A (sub)graph is *frequent* if its support in a given dataset is no less than a minimum support threshold

✓ Applications

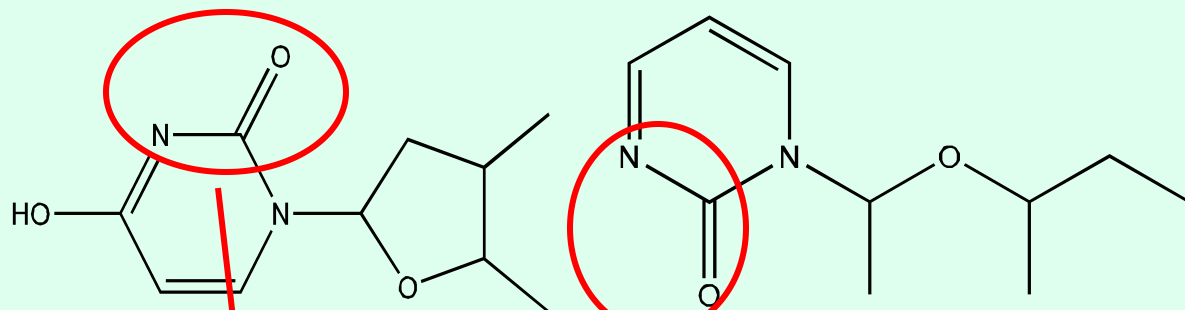
- u Mining biochemical structures
- u Mining XML structures or Web communities
- u Trees, lattices, sequences, and items are degenerated graphs

Example (Chemistry-informatics)

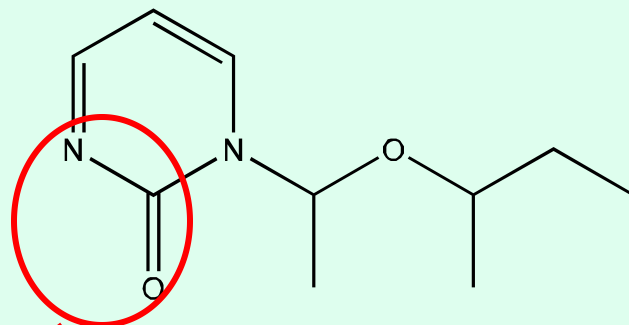
Graph Dataset



(A)

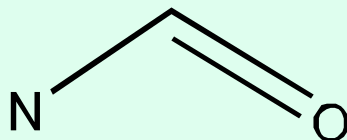


(B)

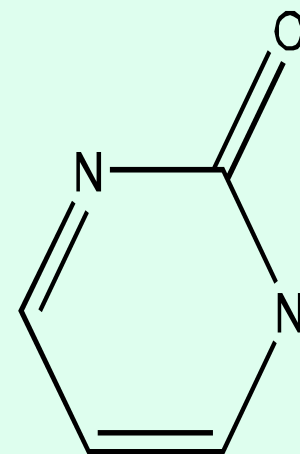


(C)

Frequent Patterns
(min support is 2)



(1)



(2)

Related Work

- ✓ **AGM/AcGM: Inokuchi, et al. (PKDD'00)**
- ✓ **FSG/FSG2: Kuramochi, et al. (ICDM'01)**
- ✓ **Vanetik, et al. (ICDM'02)**
- ✓ **Borgelt, et al. (ICDM'02)**
- ✓ **gSpan: Yan and Han (ICDM'02)**
- ✓ **...**

Subgraph Explosion Problem

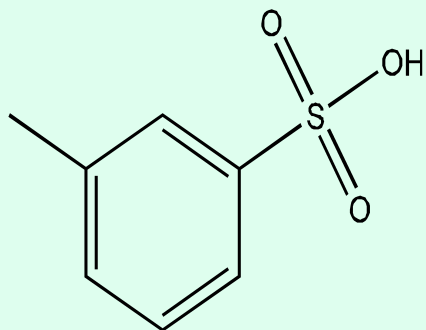
- ✓ An n -edge frequent graph may have 2^n subgraphs. All of them are frequent too.
- ✓ Among 422 chemical compounds which are confirmed to be active in an AIDS antiviral screen dataset, there are 1,000,000 frequent graph patterns if the minimum support is 5%

Observation

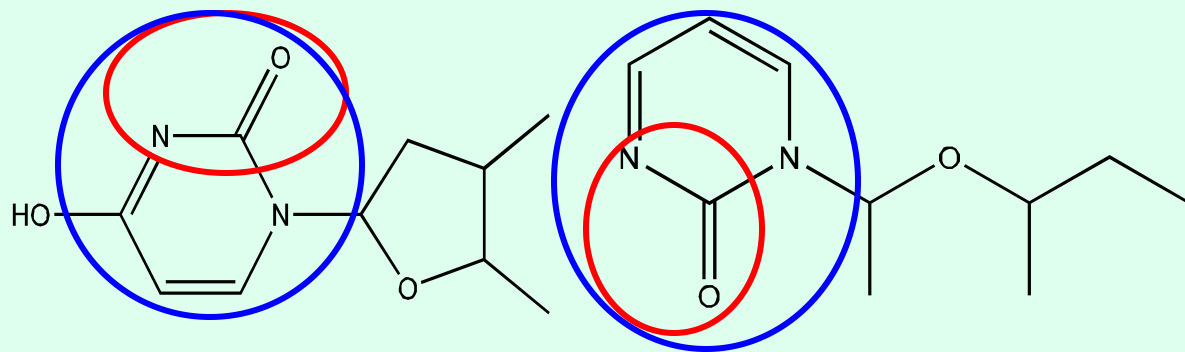
- ✓ If a graph is frequent, all of its subgraphs are frequent — **the Apriori property**
- ✓ If some of its subgraphs have the same support, it is unnecessary to output these subgraphs (**closed graphs**): This still ensures the mining result is complete—“*lossless compression*”

Example

Graph Dataset



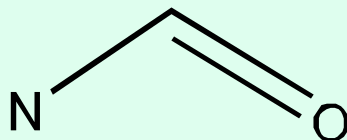
(A)



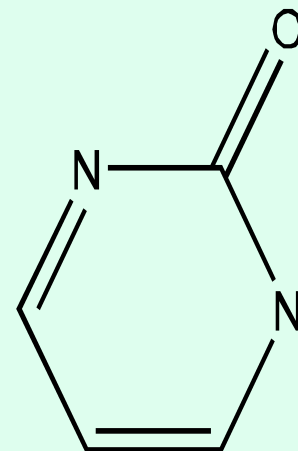
(B)

(C)

Frequent Patterns
(min support is 2)

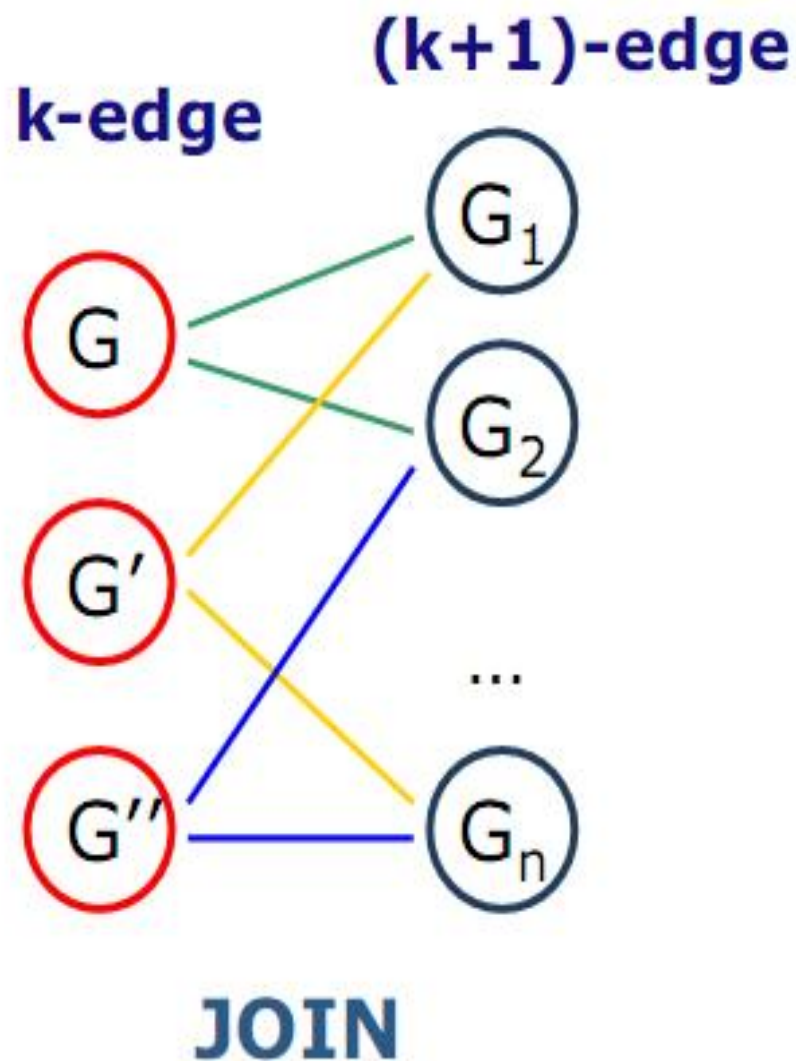


(1)



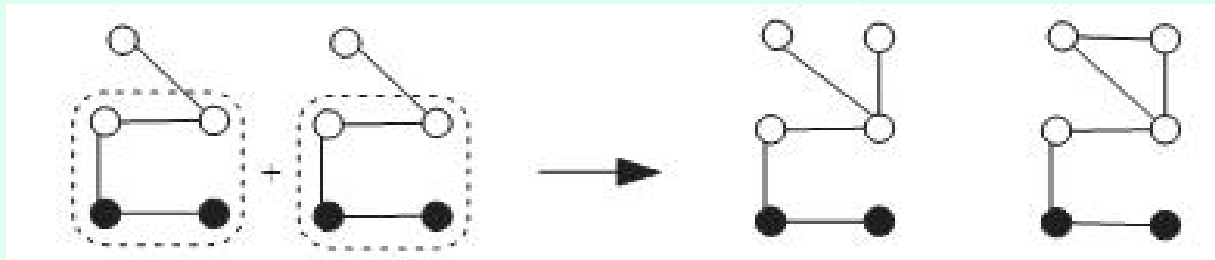
(2)

Apriori-Based Approach

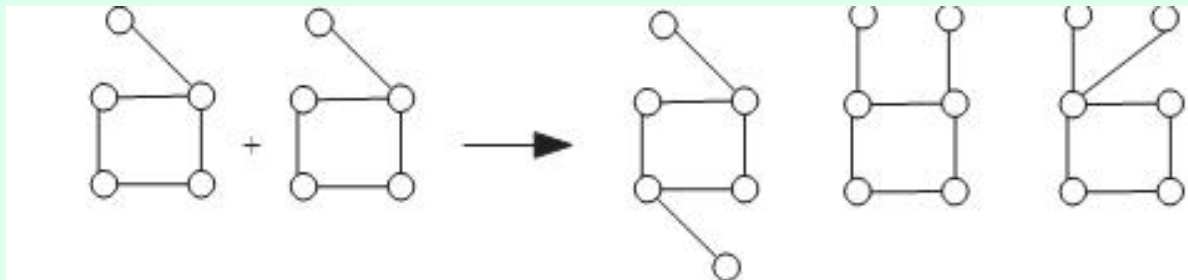


Apriori-based, BFS

- Methodology: Breadth-search, joining two graphs
- AGM (Inokuchi, et al. PKDD'00), generates new graphs with one more node



- FSG (Kuramochi and Karypis ICDM'01), generates new graphs with one more edge



Algorithm: AprioriGraph. Apriori-based frequent substructure mining.

Input:

- D , a graph data set;
- min_sup , the minimum support threshold.

Output:

- S_k , the frequent substructure set.

Method:

$S_1 \leftarrow$ frequent single-elements in the data set;

Call AprioriGraph(D, min_sup, S_1);

procedure AprioriGraph(D, min_sup, S_k)

- (1) $S_{k+1} \leftarrow \emptyset$;
- (2) for each frequent $g_i \in S_k$ do
- (3) for each frequent $g_j \in S_k$ do
- (4) for each size $(k+1)$ graph g formed by the merge of g_i and g_j do
- (5) if g is frequent in D and $g \notin S_{k+1}$ then
- (6) insert g into S_{k+1} ;
- (7) if $S_{k+1} \neq \emptyset$ then
- (8) AprioriGraph(D, min_sup, S_{k+1});
- (9) return;

So far, we covered

	Read-based	Write-based	Point-based
Association Mining	Apriori[AgSr94]	FPGrowth [HaPe00]	
Sub-Graph Mining	AGM[PKDD'00] ESG[ICDM'01]	Mofa[ICDM'02]	
Sequential Pattern Discovery	GSP[AgSr96]	PrefixSpan [PHPC01]	
Iceberg Cube	Apriori[AgSr94]		BUC[BeRa99], H-cubing [HPDW01]

References

- v [AgSr94] R. Agrawal, R. Srikant, "Fast Algorithms for Mining Association Rules", Proc. of the 20th Int'l Conference on Very Large Databases, Santiago, Chile, Sept. 1994.
- v [AgSr96] R. Srikant, R. Agrawal: "Mining Sequential Patterns: Generalizations and Performance Improvements", to appear in *Proc. of the Fifth Int'l Conference on Extending Database Technology (EDBT)*, Avignon, France, March 1996.
- v [BeRa99] Kevin S. Beyer and Raghu Ramakrishnan. "Bottom-Up Computation of Sparse and Iceberg CUBEs". In Proceedings of the ACM SIGMOD International Conference on Management of Data, pages 359-370, June 1999.
- v [HPDW01] J. Han, J. Pei, G. Dong, and K. Wang, "Efficient Computation of Iceberg Cubes with Complex Measures", Proc. 2001 ACM-SIGMOD Int. Conf. on Management of Data (SIGMOD'01), Santa Barbara, CA, May 2001.
- v [HPY00] J. Han, J. Pei, and Y. Yin, ``Mining Frequent Patterns without Candidate Generation`, Proc. 2000 ACM-SIGMOD Int. Conf. on Management of Data (SIGMOD'00), Dallas, TX, May 2000.
- v [HaPe00] J. Han and J. Pei ``Mining Frequent Patterns by Pattern-Growth: Methodology and Implications`, ACM SIGKDD Explorations (Special Issue on Scalable Data Mining Algorithms), 2(2), December 2000.
- v [PHPC01] J. Pei, J. Han, H. Pinto, Q. Chen, U. Dayal, and M.-C. Hsu, "PrefixSpan: Mining Sequential Patterns Efficiently by Prefix-Projected Pattern Growth", Proc. 2001 Int. Conf. on Data Engineering (ICDE'01), Heidelberg, Germany, April 2001.

References

- ✓ [SHSB00] P. Shenoy, J. Haritsa, S. Sudarshan, G. Bhalotia, M. Bawa, D. "[Turbo-Charging Vertical Mining of Large Database](#)." Proc. 2000 ACM-SIGMOD Int. Conf. on Management of Data (SIGMOD'00), Dallas, TX, May 2000.
- ✓ [Zaki98] Mohammed J. Zaki, "[Efficient Enumeration of Frequent Sequences](#)", 7th International Conference on Information and Knowledge Management, pp 68-75, Washington DC, November 1998.
- ✓ [Zaki00] Mohammed J. Zaki, "[Scalable Algorithms for Association Mining](#)", in IEEE Transactions on Knowledge and Data Engineering, Vol. 12, No. 3, pp 372-390, May/June 2000.
- ✓ [Zaki01] Mohammed J. Zaki, "[SPADE: An Efficient Algorithm for Mining Frequent Sequences](#)", in Machine Learning Journal, special issue on Unsupervised Learning (Doug Fisher, ed.), pp 31-60, Vol. 42 Nos. 1/2, Jan/Feb 2001.
- ✓ An Apriori-based Algorithm for Mining Frequent Substructures from Graph Data
- ✓ Frequent Subgraph Discovery